



**UNITY UNIVERSITY ADAMA CAMPUS
UNDER GRADUATE STUDIES**

**DEPARTMENT OF COMPUTER SCIENCE
DESIGN AND IMPLEMENTION OF QATAN E-LEARNING SYSTEM**

By

Group Members Here

**Nathan Zerfu
Samuel Asefa
Esmael Ali
Dawit Ayele
Hermela Genanaw
Edom Nigussie**

***June, 2025
Adama, Ethiopia***

***A FINAL PROJECT SUBMITTED TO DEPARTMENT OF COMPUTER
SCIENCE OF UNITY UNIVERSITY, ADAMA CAMPUS***

***IN PARTIAL FULFILMENT OF THE REQUIREMENT OF THE
AWARD OF BACHELOR OF SCIENCE DEGREE IN COMPUTER
SCIENCE***

Approved By Board of Examiners:

1. _____	_____	_____
<i>Chairman, Dept. Graduate Committee</i>	<i>Signature</i>	<i>Date</i>
2. _____	_____	_____
<i>Advisor</i>	<i>Signature</i>	<i>Date</i>
3. _____	_____	_____
<i>Internal Examiner</i>	<i>Signature</i>	<i>Date</i>

Acknowledgments

First and foremost, we would like to express sincere gratitude to the Almighty for His boundless grace and guidance throughout this endeavor. Without His blessings and support, none of this would have been possible, we would also thank our families, and we would also like to extend our heartfelt appreciation to our advisor, Abdu Mohammed, whose expertise, encouragement, and unwavering commitment have been invaluable throughout the development of this project. Additionally, we would like to extend our deepest gratitude to our families for their endless support and understanding.

Table of Contents

Acknowledgments	3
List of Tables	6
List of Figures	8
List of Acronyms	8
CHAPTER ONE: INTRODUCTION	9
1.1 Background	9
1.2 Statement of the Problem	9
1.3 Purpose of the Project	10
1.4 Objectives	10
1.4.1 General Objective	10
1.4.2 Specific Objectives	10
1.5 Feasibility Studies	11
1.5.1 Technical Feasibility	11
1.5.2 Operational Feasibility	11
1.5.3 Economical Feasibility	11
1.5.4 Social Feasibility	12
1.6 Scope	12
1.6.2 Limitation	12
1.7 Significance	13
1.8 Methodology	14
1.8.1 Fact-Finding Methodology	14
1.8.2 Software Development Methodology	14
1.9 Development Tools	15
1.10.1 Unit Test	16
1.10.2 Integration Test	18
1.11 Overview of Project Phases	18
1.11.1 Schedule	19
1.11.2 Team Composition	20
Chapter Two: Overview of Current System	21
2.1 Overview of Current System	21
2.1 Functions of Existing System	21
2.2 Problems of Existing System	22
2.4 Players of Existing System	22

2.5 Supplementary Specification 2.5.1 Business Rules	23
2.6 Modeling the Existing System	24
2.6.1 Use Cases	24
2.6.2 Essential Use Case Diagram	25
2.6.3 Essential Use Case Documentation	26
2.6.4 Essential Flow Diagram	34
Chapter 3 Proposed system	35
3.1 Overview of the Proposed New E-learning System	35
3.2. Functional requirement	36
3.3. Non-functional requirement	37
3.4.1.1. Actor Identification for the Proposed System	37
3.4.1.2 Identification of Use Cases	38
3.4.1.3 System Use Case Diagram	39
3.4.1.4 System Use Case Documentation	40
3.4.2. Static Model	45
3.4.2.1. Class Diagram	45
3.4.2.2. Object Diagram	46
3.4.3. Dynamic Model	47
3.4.3.1. Sequence Diagram	47
3.4.3.2 Activity Diagram	48
3.4.3.3. State Diagram	49
Chapter 4 System Design	50
4.1 Overview of System Design	50
4.2 Purpose of the System Design	50
4.3 Design Goal	50
4.4 System Architecture	51
4.5 System Process	52
4.6 System Decomposition	52
4.6.1. Subsystem Decomposition	54
4.7 Persistent Data Management	55
4.8 Component Modeling	56
4.9 Deployment Modeling	57
4.10. Database design	58
4.10.1. Database Design Overview	58
4.10.3. Logical Database Design (Relational Model)	59

4.10.4. Physical Database Design	60
4.11 Access Control for an E-Learning Platform	62
4.12. User interface design	63
CHAPTER FIVE: IMPLEMENTATION AND TESTING	66
5.1 Introduction	66
5.2 Coding	66
5.2.1 Coding Style	66
5.2.2 Programming Language and Technology Selection	67
5.3 Software Testing and Procedures	68
5.3.1 Software Testing	68
5.3.2 Testing Procedures	69
CHAPTER SIX: SUMMARY, CONCLUSION AND RECOMMENDATION	70
6.1 Summary and Conclusion	70
6.2 Recommendation	70
4.13. Reference	71
Appendixes	72

List of Tables

Table 1.1 Project Phase	18
Table 1.2. Project Team Organization	20
Table 2.1 Players of Existing System	22
Table 2.2 Use Cases	24
Table 2.3 Attend In-person Classes Use Case Documentation	26

Table 2.4 Manually Submit Assignments Use Case Documentation	26
Table 2.5 Register for Courses Use Case Documentation	26
Table 2.6 Enroll for Courses Use Case Documentation	27
Table 2.7 Make Tuition Payments Use Case Documentation	27
Table 2.8 Deliver Lectures Use Case Documentation	27
Table 2.9 Share Materials (via Telegram) Use Case Documentation	28
Table 2.10 Assign Assignments Use Case Documentation	28
Table 2.11 Manually Grade Assignments Use Case Documentation	28
Table 2.12 Administer Examinations Use Case Documentation	29
Table 2.13 Coordinate Academic Activities Use Case Documentation	29
Table 2.14 Ensure Curriculum Adherence Use Case Documentation	29
Table 2.15 Schedule Exams Use Case Documentation	30
Table 2.16 Supervise Department Operations Use Case Documentation	30
Table 2.17 Approve Course Plans Use Case Documentation	30
Table 2.18 Manage Faculty Performance Use Case Documentation	31
Table 2.19 Oversee Faculty-Level Academic Affairs Use Case Documentation	31
Table 2.20 Enforce University Policies and Quality Standards Use Case Documentation	31
Table 2.21 Manage Student Registration Use Case Documentation	32
Table 2.22 Maintain Academic Records Use Case Documentation	32
Table 2.23 Schedule Classes and Exams Use Case Documentation	32
Table 2.24 Handle Student Tuition Payments Use Case Documentation	33
Table 2.25 Issue Payment Receipts Use Case Documentation	33
Table 2.26 Manage Financial Records Use Case Documentation	33
Table 3.1 Identification of Actors	38
Table 3.2 Identification of Use Cases	38
Table 3.3 User Registration	40
Table 3.4 User Login	40
Table 3.5 Create Course	41
Table 3.6 Enroll in Course	41
Table 3.7 Upload Course Materials	41
Table 3.8 Access Course Materials	41
Table 3.9 Create Quiz	42
Table 3.10 Submit Take Quiz	42
Table 3.11 Track Student Performance	43
Table 3.12 Generate Reports	43
Table 3.13 Communication via Discord	43
Table 3.14 Manage Enrollment	44
Table 3.15 Process Payment	44
Table 3.16 View Course Catalog	44
Table 4.1 Physical Database Design	60

List of Figures

Figure 1.1. Time Management	19
Figure 2.1 Essential Use Case Diagram	25
Figure 2.1 Essential Flow Diagram for Existing Diagram	34
Figure 3.1. Use case diagram	39
Figure 3.2. Class Diagram	45
Figure 3.3. Object Diagram Student Enrolled in a Course	46
Figure 3.4. Sequence Diagram	47
Figure 3.5. Activity Diagram	48
Figure 3.6. State chart Diagram	49
Figure 4.1. Interaction of Layers	52
Figure 4.2. Subsystem Decomposition	54
Figure 4.3. Persistent Data Management diagram	55
Figure 4.4. Component Modeling	56
Figure 4.5. Collaboration diagram	57
Figure 4.6. Conceptual Database Design	58
Figure 4.7. Logical Database Design	59
Figure 4.8. Data Representation Design	61
Figure 4.9 Homepage	63
Figure 4.10 Course Page	64
Figure 4.11 about Us Page	65

List of Acronyms

API – Application Programming Interface
CRUD – Create, Read, Update, Delete
CSS – Cascading Style Sheets
DB – Database
DM – Direct Message
ERD – Entity Relationship Diagram
IDE – Integrated Development Environment
JS – JavaScript
QA – Quality Assurance
RBAC – Role-Based Access Control
PostgreSQL– Structured Query Language
UI – User Interface
UX – User Experience

CHAPTER ONE: INTRODUCTION

1.1 Background

The landscape of higher education globally has historically been dominated by traditional, brick-and-mortar institutions where learning primarily occurs through face-to-face interaction in physical classrooms with fixed schedules. This model, while having a long-standing history of success, inherently presents limitations in the modern era. Students often face challenges related to geographical barriers, the rigidity of timetables that may conflict with work or personal commitments, and restricted access to educational resources outside of designated hours. This can particularly impact non-traditional students or those in remote areas.

Focusing on the context of private universities, institutions like Unity University in Ethiopia operate within this traditional framework and face similar challenges. Students at such universities navigate issues such as commuting time, the inability to revisit complex lecture content easily, and the challenge of accommodating diverse learning paces within a standardized classroom setting. The reliance on physical infrastructure also places limits on the number of students that can be effectively served and the flexibility of program delivery.

The advent and evolution of e-learning systems offer a compelling alternative and supplementary approach. E-learning leverages digital technologies to deliver educational content and facilitate interaction online. Such a system has the potential to directly address the aforementioned limitations faced by traditional private universities by providing flexible access to learning materials anytime, anywhere, accommodating varied schedules, and offering opportunities for self-paced learning. Developing a standalone web-based e-learning system could therefore serve as a transformative solution to enhance accessibility, improve resource utilization, and provide a more adaptable learning environment for students attending institutions like Unity University.

1.2 Statement of the Problem

The current traditional educational model, as implemented in many private universities exemplified by institutions like Unity University, faces significant challenges that impede optimal student engagement and educational delivery. These problems, stemming from the inherent structure of traditional learning, manifest as:

- **Inflexibility of Access:** Rigid class schedules and the requirement for physical attendance create significant barriers for students who are working, have family responsibilities, or reside far from the university campus, limiting their access to education.

- **Limited and Inflexible Resource Availability:** Students often have restricted access to lecture materials, supplementary readings, and other learning resources, primarily confined to class time or limited library hours, hindering their ability to review and reinforce learning at their own pace.
- **Constraints on Scalability and Reach:** The capacity of traditional universities to enroll more students and offer programs to a wider geographical area is limited by the physical capacity of classrooms and facilities.
- **Inability to Cater to Diverse Learning Paces:** The standardized pace of traditional lectures does not effectively support students who may need more time to understand concepts or those who are capable of progressing more quickly, potentially leading to disengagement or incomplete understanding
- **Lack of Opportunity for Repeated Exposure:** Students may find it difficult to revisit lectures or explanations multiple times, which is often necessary for mastering complex subjects, unlike the potential for on-demand access provided by digital platforms.

1.3 Purpose of the Project

The overall purpose of this project is to design and implement a functional and accessible standalone web-based e-learning system. This undertaking is crucial because it directly addresses the limitations of the current traditional learning model prevalent in private universities, aiming to provide a flexible, accessible, and efficient alternative for delivering education. The importance of this project lies in its potential to offer a modern, scalable, and adaptable learning platform that can enhance the educational experience for students who might otherwise face barriers in a purely traditional setting.

1.4 Objectives

1.4.1 General Objective

To develop and deploy a comprehensive, standalone web-based e-learning platform that provides flexible, accessible, and effective online learning opportunities for students.

1.4.2 Specific Objectives

- To enable students to access course materials, lectures, and resources online at any time and from any location with internet access.
- To facilitate flexible learning schedules that accommodate students' varied commitments.
- To provide tools for students to learn at their own pace and revisit content as needed.
- To create a platform that supports interactive learning and communication between students and instructors online.
- To offer a scalable solution that can potentially serve a large and geographically dispersed students.

- To improve the overall educational experience and outcomes for students by leveraging digital technologies in an accessible platform.

1.5 Feasibility Studies

We will conduct a comprehensive feasibility study to determine the viability of developing and implementing this standalone web-based e-learning system.

1.5.1 Technical Feasibility

We will assess the technical requirements for developing and hosting a robust web-based e-learning platform. This will involve evaluating the necessary software architecture, database design, hosting infrastructure (servers, bandwidth), and integration capabilities with potential third-party services (e.g., payment gateways). We will identify the features we will implement, such as:

- User authentication and profile management (students, instructors, administrators).
- Course creation and content management (uploading photo and videos).
- Discussion forums and communication tools.
- Online quizzes and grading.
- Progress tracking and reporting.
- Payment processing for course fees (if applicable).

1.5.2 Operational Feasibility

We will evaluate the operational requirements for running and maintaining the standalone e-learning system. This will include identifying the necessary human resources for technical support, content management, and administration. We will detail the workflows for onboarding instructors and students, managing courses, and handling technical issues. We anticipate that the new system will streamline educational delivery processes, reduce administrative overhead related to physical classrooms, and provide a more efficient platform for managing student data and interactions compared to purely manual or traditional methods.

1.5.3 Economical Feasibility

We will estimate the financial resources required for the entire project lifecycle, including development costs, infrastructure (hosting), implementation, marketing, and ongoing maintenance and support. We will assess the potential revenue streams e.g., course revenue and compare them against the estimated costs to determine the project's economic viability and potential return on investment. This will involve creating a detailed budget and exploring potential funding models.

1.5.4 Social Feasibility

We will evaluate the social acceptability and potential adoption of a standalone e-learning platform among the target user base (students seeking flexible education) and potential content providers (instructors or institutions). This will involve assessing the demand for online learning, identifying potential barriers to adoption (e.g., trust in online platforms, preference for traditional methods), and developing strategies to build credibility and encourage participation. The project is expected to be socially acceptable as it aligns with the growing global acceptance of online education and offers benefits such as increased access and flexibility that are likely to be welcomed by a significant segment of the student population.

1.6 Scope

The system will provide core functionalities for online course delivery and management. The key features and requirements of the software will include secure user accounts for students, instructors, and administrators, a content management system for uploading, organizing, and delivering various types of course materials (Photo and video), and tools for creating and managing quizzes. Furthermore, the platform will incorporate communication features such as discord server. It will also feature a system for tracking student progress, performance, and generating reports, a mechanism for handling course enrollment and payment processing (if possible), and a responsive design to ensure accessibility and usability on various devices (desktops, tablets, smartphones).

1.6.2 Limitation

Based on common limitations observed in standalone e-learning platforms and the target environment, this project may face the following limitations:

- **Internet Connectivity Dependence:** The platform's usability is heavily reliant on students and instructors having stable and affordable internet access, which can be a significant challenge in some regions.
- **User Digital Literacy and Equipment:** Varying levels of digital literacy among potential users and unequal access to suitable computing devices can hinder adoption and effective use.
- **Technical Support Scalability:** Providing timely and effective technical support to a large, geographically dispersed user base can be operationally complex and resource-intensive.
- **Content Quality and Development:** The availability and creation of high-quality, engaging digital course content is crucial for the platform's success but can require significant effort and expertise.
- **Building Trust and Credibility:** As a new, standalone platform, establishing trust and credibility among potential users and institutions may take time and dedicated effort.

- **Competition:** The e-learning market is competitive, and attracting and retaining users may require ongoing innovation and marketing efforts.
- **Initial Development and Marketing Cost:** The initial investment required for developing a robust platform and effectively marketing it to attract users can be substantial.
- **Lack of Advanced Interactive Features:** The current scope focuses on core content delivery and basic communication. Features like live virtual classrooms, real-time collaborative tools, or advanced interactive simulations are not included, limiting the platform's ability to replicate dynamic in-person interactions.
- **No Personalized Learning Paths:** The system does not include capabilities for personalized learning paths or adaptive content delivery based on individual student performance and learning styles, which could optimize learning outcomes.
- **Absence of Dedicated Mobile Applications:** While the web platform will be responsive, the scope does not include the development of native mobile applications for iOS or Android, which could offer a more tailored and potentially richer user experience on mobile devices, including offline access to content.

1.7 Significance

Upon successful delivery and adoption, this e-learning platform is expected to bring about significant positive changes and outcomes:

- **Increased Educational Access:** The platform will provide a new avenue for individuals to access educational content and courses regardless of their geographical location or time constraints.
- **Democratization of Education:** By providing an accessible online option, the platform can help democratize access to quality educational resources.
- **Enhanced Learning Flexibility:** Users will benefit from flexible learning schedules and the ability to learn at their own pace, potentially leading to improved understanding and knowledge retention.
- **Support for Traditional Institutions:** The platform can serve as a valuable tool for traditional institutions (like private universities) to supplement their offerings, reach new student demographics, or transition to blended learning models.
- **Promotion of Digital Literacy:** Use of the platform will inherently promote digital literacy skills among its users.
- **Potential for Diverse Course Offerings:** The platform can host a wide variety of courses from different subject areas and content providers.

1.8 Methodology

1.8.1 Fact-Finding Methodology

To gain a comprehensive understanding of the core challenges faced by learners and educators in traditional learning environments, several fact-finding techniques were employed. These methods provided critical insights that guided the development of the proposed e-learning system, ensuring it responds effectively to user needs.

- **Student Interviews:** Interviews conducted with students revealed recurring issues such as difficulty attending scheduled classes due to commuting challenges, work obligations, and personal responsibilities. Additionally, students expressed frustration over the inability to revisit lectures, which hindered their understanding of complex subjects.
- **Document Review:** An analysis of existing academic documents highlighted inefficiencies in traditional academic management practices. Key processes such as grade reporting, attendance tracking, and assignment submission were often manual, leading to errors and delays.
- **Observations:** Direct observation of classroom and administrative activities confirmed that a significant portion of instructional and operational time was consumed by repetitive tasks, such as distributing printed materials, taking attendance manually, and collecting assignments. These observations emphasized the need for automation and centralized digital systems.
- Together, these findings provided a strong foundation for the system's design, ensuring that the proposed e-learning platform addresses the key pain points associated with conventional education delivery.)

1.8.2 Software Development Methodology

We chose the Agile Software Development Methodology for this project because it provides flexibility, supports frequent updates, and encourages ongoing collaboration.

Agile allowed us to break the work into manageable parts and prioritize the most important features first. It helped us test early modules (like registration and login) and improve them based on team feedback before moving forward. Using Agile also made it easier to coordinate as a team, including through online communication tools like Telegram. In short, Agile was selected because it helped us stay organized, respond to changes quickly, and continuously improve the system throughout the project timeline.

1.9 Development Tools

- **Visual Studio Code:**-Used as the primary source code editor for writing and managing React, CSS, Tailwind CSS JavaScript, and node JS code for the web application.
- **Vercel (Hosted Environment):** - Provides a fast, scalable, and server less deployment platform used to host the web application online.
- **Node JS:** -Utilized as the server-side scripting language for handling business logic, database interactions, and dynamic content.
- **React, Tailwind CSS, and JavaScript:** -Front-end technologies used to create the structure, design, and interactivity of the e-learning platform accessible via web browsers.
- **Microsoft Word:** -Used to create, format, modify, and print the documentation of the developed system.
- **Microsoft Project:** -Employed to schedule and track the project timeline, tasks, and milestones for system development.
- **Microsoft PowerPoint:** -Used to prepare and deliver project presentations, including the final defense.
- **AJAX:** -Used to send and receive data from the server asynchronously without reloading the page.
- **Google Chrome:** -Primary web browser used to test and debug the system's front-end and back-end.
- **PostgreSQL:** -Relational database used to store user data, course content, assignments, and grades.
- **Node.js Local Server:** A local development server used to run and test the backend API during the system's development.
- **Prisma Studio:** Browser-based GUI tool used to visually browse, edit, and manage the PostgreSQL database through Prisma during development.
- **GitHub:** -Version control and collaborative platform for storing, sharing, and managing the code.
- **Telegram:** -Used for continuous group communication through chats and video calls to coordinate project discussions and collaboration.

1.10.1 Unit Test

Unit testing will entail verifying the individual components or modules of the system in isolation. Each module will be tested individually to verify that it performs its desired function in the right manner and processes various inputs in the anticipated fashion.

Modules to be Tested and Testing Method:

User Authentication and Profile Management Module:

- **Functionality:** Verify user registration, login, logout, password reset, and profile update functionalities.
- **Testing Strategy:** Enter valid and invalid credentials, test for proper error messages, and ensure that session management is secure.
- **Worst-case scenarios:** Attempting login with non-existent users, SQL injection attacks in input fields, and concurrent login attempts from different geographical locations.
- **Expected good outputs:** Successful login/logout, proper profile updates, appropriate error messages for improper inputs.

Course Creation and Content Management Module

- **Functionality:** Validate uploading, organizing, and delivering capacity for various course content types (video, images).
- **Testing Method:** Upload various file sizes and types, editing and deleting content testing, and verifying content display.
- **Worst-case cases:** Damaged file uploads, exceeding file size restrictions, attempting deleting content with improper permissions.
- **Expected good outputs:** Display and uploading of diverse content, effective content organization, proper content updates.

Discord Communication Tools Module

Functionality:

- Test the creation of discussion channels, posting messages, and direct messaging between users.

Testing Method:

1. Create New Channels: Set up various text channels for different topics (e.g., course discussions, announcements).

2. Post Messages:

- Create new threads within channels to facilitate focused discussions.
- Post replies to existing threads.

3. Direct Messaging: Send private messages between different user roles (e.g., student-instructor, student-student).

Worst-case Scenarios:

- Attempt to post extremely lengthy messages that may exceed character limits.
- Try to post in channels that the user does not have access to (e.g., restricted channels).
- Send direct messages to users who are not on the server or have DMs disabled.

Expected Good Outputs:

- Functional discussion channels with clear organization.
- Instant delivery of messages and replies within channels and direct messages.
- Proper message history available for all discussions, allowing users to refer back to previous conversations.

Online Quizzes Module:

- Functionality: Test the creation, administration, and scoring of online quizzes.
- Testing Strategy: Try out quizzes with correct and incorrect answers, test time limits, and verify automatic grading.
- Worst-case situations: Submitting quizzes after deadline, attempting to re-take a locked-out quiz, system failure in the middle of a quiz.
- Good outputs to be expected: Accurate quiz results, correct score calculation, correct feedback to answers.

Progress Tracking and Reporting Module:

- Functionality: Test tracking student progress and reporting.
- Test Approach: Verify progress updates at the time of lesson or Quizz completion, and test reports for correctness generated for students and educators.
- Worst-case scenarios: Erroneous progress updates, inability of reports to be generated as a result of data inconsistency, unauthorized access attempt to reports.
- Expected good outputs: Tracking of progress in real-time and accurately, proper generation of performance reports.

Payment Processing Module (if it applies):

- Functionality: Test processing of course fees.
- Testing Methodology: Simulate successful and failed payment transactions, test multiple payment types.

- Worst-case scenarios: Payment gateway crash, making attempt to process fraudulent payment, processing wrong payment amount.
- Expected good outcomes: Payment success, accurate transaction recording, correct error handling on failed payments.

1.10.2 Integration Test

We will integrate the following modules and test their interactions:

- User Authentication + User Dashboard
- Course Management + Quiz Module
- Quiz Module + Progress Tracker
- Course Management
- Discord Communication module

Integration testing will ensure correct data flow, proper interaction between components, and consistent system behavior.

1.11 Overview of Project Phases

The project will be divided based on specific development activities aligned with the agile methodology:

Phase Description	Key Activities
Requirement Gathering & Analysis	Interviews, document reviews, classroom observations
System Design	UI/UX design, system architecture, database schema design
Iterative Module Development	Feature-wise development of core modules like registration, course management
System Integration	Combine modules, ensure data consistency across components
Testing	Perform unit tests, integration tests, fix bugs
Deployment & Hosting	Deploy the platform to a web hosting server

Table 1.1 Project Phase

1.11.1 Schedule

The following table represents the Gantt chart-style schedule:

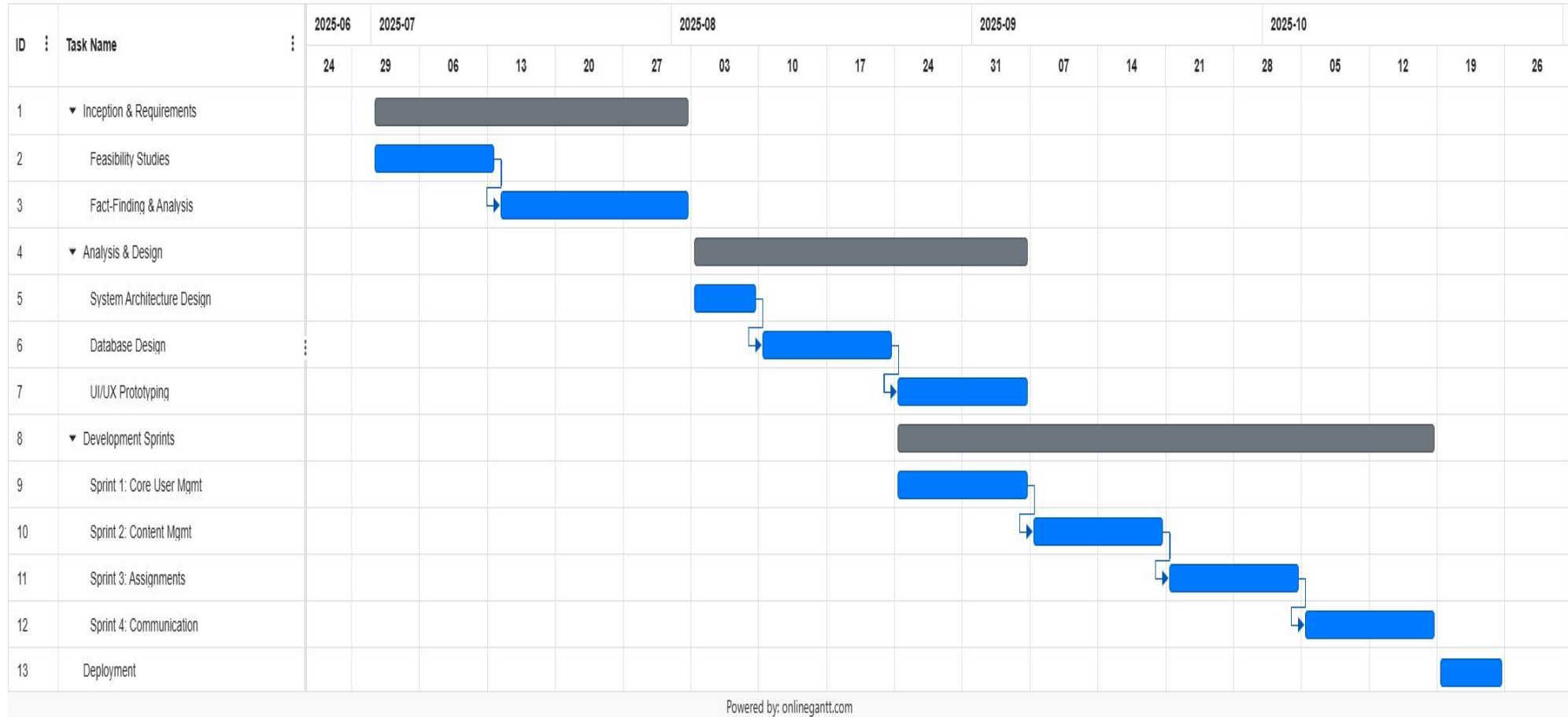


Figure 1.1. Time Management

1.11.2 Team Composition

Team Member (Role)	Key Responsibilities	Team Member
Project Manager	Overall project planning, coordination, risk management, stakeholder communication, schedule tracking	Nathan
Lead Developer (Backend)	Design and implement server-side logic (Node JS), database interactions, API development, and system architecture.	Samuel
Frontend Developer	Develop user interface (React, tailwind CSS, and JavaScript), ensure responsive design, UI/UX implementation.	Esamel
Database Administrator/Developer	Design and manage the database schema, ensure data integrity and security, optimize database queries.	Hermela
Quality Assurance (QA) Tester	Develop and execute test plans (unit and integration tests), identify and report bugs, ensure software quality	Edom
Technical Writer/Documenter	Create and maintain all project documentation (e.g., system design, user manuals)	Dawit

Table 1.2. Project Team Organization

Chapter Two: Overview of Current System

2.1 Overview of Current System

The current educational system at Unity University operates primarily through a traditional, manual, and physical classroom model. Its functions involve face-to-face lecture delivery, manual attendance and assignment handling, and paper-based student record keeping. Key players include students, instructors, and administrative staff. This system faces problems such as rigid scheduling, limited access to resources, and inefficiencies from its manual nature. For example, business rules enforce mandatory physical attendance and strict physical assignment deadlines. Modeling this system would reveal its bottlenecks and limited scalability due to heavy reliance on physical interactions and manual information transfer.

2.1 Functions of Existing System

The functions of the existing traditional system, identified during fact-finding, are primarily manual and physical in nature:

- **Lecture Delivery:** Instructors conduct classes through in-person, synchronous lectures in physical classrooms.
- **Attendance Tracking:** Student attendance is recorded manually, typically via paper sign-in sheets or roll calls by instructors.
- **Assignment Distribution and Collection:** Assignments are distributed as physical handouts and collected as hard copies.
- **Assignment Grading and Feedback:** Instructors manually grade physical assignments and provide feedback in written form.
- **Examination Administration:** Quizzes and exams are conducted in a supervised physical setting, followed by manual grading.
- **Student Registration and Enrollment:** Students register for courses and enroll through manual processes involving forms and administrative staff.
- **Record Keeping:** Student grades, attendance, and personal information are primarily maintained in physical files or basic, non-integrated digital spreadsheets by administrative departments.
- **Communication:** Interaction between students and instructors occurs predominantly face-to-face during class hours or office visits.

2.2 Problems of Existing System

During the requirements elicitation phase, several key issues were identified in the existing traditional learning system used by private universities such as Unity University. These problems include:

- **Rigid Class Schedules:** The system requires students to attend scheduled classes in person, making it difficult for those with jobs, family responsibilities, or long commutes.
- **No On-Demand Lecture Replay:** Students are unable to revisit lectures or explanations at their own pace, making it harder to understand complex topics.
- **Manual Assignment Submission and Grading:** Submitting assignments physically and grading them manually results in delays, errors, and inefficiencies.
- **Low Scalability:** The traditional system relies on physical infrastructure, limiting its capacity to accommodate increasing numbers of students.
- **No Real-Time Progress Tracking:** Students and instructors lack tools to monitor learning progress, performance, or engagement levels in real-time.
- **High Administrative Workload:** Administrative tasks like taking attendance, managing grades, and distributing materials are time-consuming and prone to human error.
- **Difficulty in Catering to Different Learning Paces:** The system does not support learners who prefer to advance faster or require more time, leading to gaps in understanding.
- **No Centralized Data Management:** Academic records, student profiles, and course documents are often managed manually or across disconnected systems, causing redundancy and inefficiencies.
- **No Digital Backup of Academic Data:** Paper-based systems are vulnerable to loss, damage, and unauthorized access due to lack of digital storage and security protocols.

2.4 Players of Existing System

Player (Actor)	Responsibilities
Students	Attend in-person classes, receive materials via Telegram, and manually submit assignments.
Instructors	Deliver lectures, share materials via Telegram, assign and manually grade assignments.
Department Coordinators	Coordinate academic activities, ensure curriculum adherence, and schedule exams.
Department Heads	Supervise department operations, approve course plans, and manage faculty performance.
Dean's Office	Oversee faculty-level academic affairs, enforce university policies and quality standards.
Registrar Office	Manage student registration, maintain academic records, schedule classes and exams.
Finance Office	Handle student tuition payments, issue payment receipts, and manage financial records.

Table 2.1 Players of Existing System

2.5 Supplementary Specification

2.5.1 Business Rules

The existing traditional educational system operates based on several implicit and explicit business rules that govern its procedures and interactions. These rules, derived from the current operational model, include:

- **Physical Attendance Requirement:** Students are required to physically attend scheduled classes in designated classrooms to be marked present and receive instruction.
- **Manual Attendance Recording:** Instructors are responsible for manually tracking student attendance, typically via paper sign-in sheets or roll calls.
- **Physical Assignment Submission:** All assignments must be submitted by students as hard copies, directly to the instructor or a designated collection point.
- **Manual Assignment Grading:** Instructors manually grade physical assignments and provide feedback in written form on the submitted hard copies.
- **In-Person Examination Administration:** Quizzes and exams are conducted in a supervised physical setting, requiring students to be present at a specific time and location.
- **Paper-Based Record Keeping:** Student grades, attendance records, and personal information are primarily maintained in physical files or basic, non-integrated digital spreadsheets by administrative departments.
- **Face-to-Face Communication Protocol:** Primary communication between students and instructors occurs through direct, in-person interaction during class hours or scheduled office visits.
- **Fixed Lecture Schedule Adherence:** Lectures are delivered synchronously according to a predetermined timetable, and students must adhere to this schedule to receive instruction.

2.6 Modeling the Existing System

2.6.1 Use Cases

Here is a list of the use cases for the existing system, formatted as requested:

No	Use Case ID	User Registration
1	UC – 01	Attend In-person Classes
2	UC – 02	Manually Submit Assignments
3	UC – 03	Register for Courses
4	UC – 04	Enroll for Courses
5	UC – 05	Make Tuition Payments
6	UC – 06	Deliver Lectures
7	UC – 07	Share Materials (via Telegram)
8	UC – 08	Assign Assignments
9	UC – 09	Manually Grade Assignments
10	UC – 10	Administer Examinations
11	UC – 11	Coordinate Academic Activities
12	UC – 12	Ensure Curriculum Adherence
13	UC – 13	Schedule Exams
14	UC – 14	Supervise Department Operations
15	UC – 15	Approve Course Plans
16	UC – 16	Manage Faculty Performance
17	UC – 17	Oversee Faculty-Level Academic Affairs
18	UC – 18	Enforce University Policies and Quality Standards
19	UC – 19	Manage Student Registration
20	UC – 20	Maintain Academic Records
21	UC – 21	Schedule Classes and Exams
22	UC – 22	Handle Student Tuition Payments
23	UC – 23	Issue Payment Receipts
24	UC – 24	Manage Financial Records

Table 2.2 Use Cases

2.6.2 Essential Use Case Diagram

The following figure shows the Essential Use Case Diagram of Existing University System that shows the interaction between the players and the functionalities of the manual System.

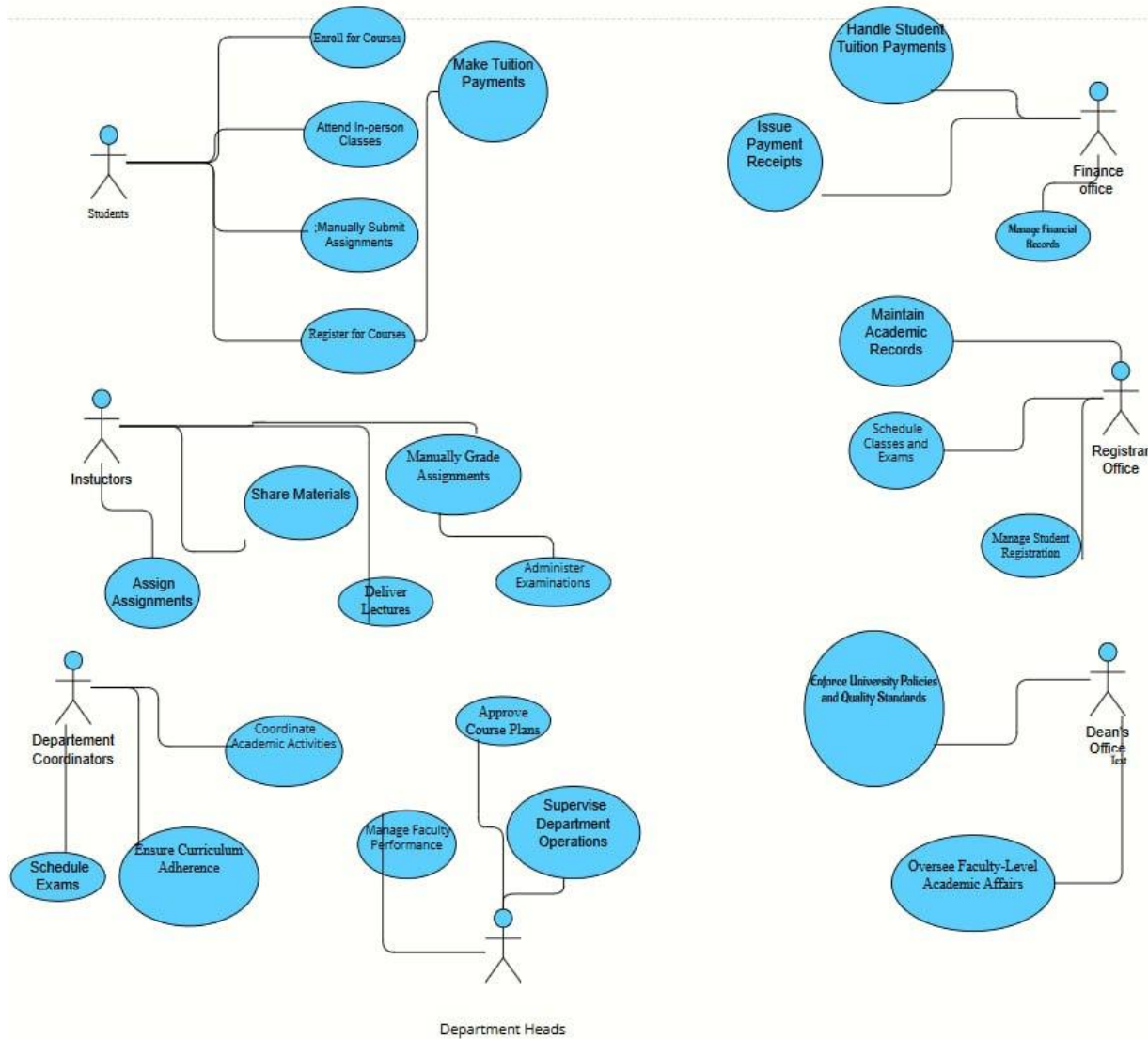


Figure 2.1 Essential Use Case Diagram

2.6.3 Essential Use Case Documentation

Identifier: UC-01
Description: Describes how a student attends a scheduled class physically.
Actor: Student
Precondition: Student is enrolled in the course.
Post condition: Attendance is marked (manually or otherwise).
Basic Course of Action: 1. Student checks class schedule. 2. Student arrives at the classroom on time. 3. Instructor begins the lecture. 4. Student attends the class. 5. Instructor marks attendance.
Alternative Course of Action: 1. Student misses class and must follow up with instructor or peers

Table 2.3 Attend In-person Classes Use Case Documentation

Identifier: UC-02
Description: Student submits their assignment in person or via non-digital means.
Actor: Student
Precondition: Student has received assignment and completed it.
Post condition: Instructor receives and stores assignment.
Basic Course of Action: 1. Student completes assignment. 2. Student physically submits assignment to instructor or office. 3. Instructor stores the submission.
Alternative Course of Action: 1. Assignment is submitted late; student may face penalties.

Table 2.4 Manually Submit Assignments Use Case Documentation

Identifier: UC-03
Description: Student registers for academic courses.
Actor: Student
Precondition: Registration period is open.
Post condition: Student is officially registered in selected courses
Basic Course of Action: 2. Student obtains course list and registration form. 3. Student selects courses and fills the form. 4. Student submits form to registrar. Registrar confirms registration.
Alternative Course of Action: Course is full; student must select a different course.

Table 2.5 Register for Courses Use Case Documentation

Identifier: UC-04
Description: Student formally enrolls in a course after registration.
Actor: Student
Precondition: Student is registered and tuition is paid.
Post condition: Enrolment is recorded.
Basic Course of Action: 1. Student confirms enrolment with registrar. 2. Enrollment is recorded.
Alternative Course of Action: 1. Student's payment is incomplete; Enrollment is not finalized.

Table 2.6 Enroll for Courses Use Case Documentation

Identifier: UC-05
Description: Student pays tuition fees.
Actor: Student
Precondition: Tuition amount is known.
Post condition: Payment is recorded by finance office.
Basic Course of Action: 2. Student checks balance. 3. Student goes to finance office or bank. 4. Student makes payment. 5. Finance office records payment.
Alternative Course of Action: 1. Payment is delayed; student receives a warning.

Table 2.7 Make Tuition Payments Use Case Documentation

Identifier: UC-06
Description: Instructor teaches students in a physical classroom.
Actor: Instructor
Precondition: Class is scheduled.
Post condition: Lecture is delivered.
Basic Course of Action: 2. Instructor prepares lecture. 3. Instructor arrives in class. 4. Instructor delivers lecture.
Alternative Course of Action: 1. Instructor is absent; class is postponed.

Table 2.8 Deliver Lectures Use Case Documentation

Identifier: UC-07
Description: Instructor distributes course materials through Telegram.
Actor: Instructor
Precondition: Students are enrolled and have access to Telegram.
Post condition: Students receive the materials.
Basic Course of Action: - Instructor compiles course materials. - Instructor sends files via Telegram group. - Students download or access files.
Alternative Course of Action: Students do not check messages; materials are missed.

Table 2.9 Share Materials (via Telegram) Use Case Documentation

Identifier: UC-08
Description: Instructor assigns homework or coursework.
Actor: Instructor
Precondition: Course is in session.
Post condition: Assignment is distributed to students.
Basic Course of Action: - Instructor prepares assignment. - Instructor shares it verbally or via Telegram. - Students note the instructions.
Alternative Course of Action: Assignment is not clearly understood; students ask for clarification.

Table 2.10 Assign Assignments Use Case Documentation

Identifier: UC-09
Description: Instructor reviews and grades submitted assignments manually.
Actor: Instructor
Precondition: Assignments have been submitted.
Post condition: Grades are recorded on paper or simple system.
Basic Course of Action: - Instructor reviews submission. - Assigns score based on rubric. - Notes grade manually.
Alternative Course of Action: Grade disputes may arise.

Table 2.11 Manually Grade Assignments Use Case Documentation

Identifier: UC-10
Description: Instructor tracks academic performance of students.
Actor: Instructor
Precondition: Grades and attendance data are available.
Post condition: Performance reports are generated.
Basic Course of Action: 1. Instructor collects data. 2. Analysis performance trends. 3. Identifies students needing support.
Alternative Course of Action: Insufficient data; tracking is delayed.

Table 2.12 Administer Examinations Use Case Documentation

Identifier: UC-11
Description: Administrator keeps records of student data.
Actor: Administrator
Precondition: Student information is collected.
Post condition: Records are stored and updated.
Basic Course of Action: 1. Collect personal and academic data. 2. Input into record system. 3. Update as necessary.
Alternative Course of Action: 1. Data entry errors; records must be corrected.

Table 2.13 Coordinate Academic Activities Use Case Documentation

Identifier: UC-12
Description: Registrar approves submitted course registration forms.
Actor: Registrar
Precondition: Student submitted registration form.
Post condition: Registration is approved or rejected.
Basic Course of Action: 1. Review form. 2. Approve or suggest alternatives.
Alternative Course of Action: 1. Errors on form; returned to student.

Table 2.14 Ensure Curriculum Adherence Use Case Documentation

Identifier: UC-13
Description: Administrator generates student and course-related reports.
Actor: Administrator
Precondition: Data exists in the system.
Post condition: Reports are ready for review.
Basic Course of Action: 1. Select report type. 2. Retrieve data. 3. Format and export report.
Alternative Course of Action: 1. Data is outdated or incomplete.

Table 2.15 Schedule Exams Use Case Documentation

Identifier: UC-14
Description: Students provide feedback on courses or instructors.
Actor: Student
Precondition: Course has been attended.
Post condition: Feedback is received by the institution.
Basic Course of Action: 1. Student fills out feedback form. 2. Submits to administration or instructor.
Alternative Course of Action: 1. Feedback is anonymous and not actionable.

Table 2.16 Supervise Department Operations Use Case Documentation

Identifier: UC-15
Description: Instructor or admin reviews and acts upon student feedback.
Actor: Instructor/Admin
Precondition: Feedback has been received.
Post condition: Improvements or notes are made.
Basic Course of Action: 1. Review feedback content. 2. Discuss with relevant parties. 3. Plan improvements.
Alternative Course of Action: 1. Feedback is irrelevant or vague.

Table 2.17 Approve Course Plans Use Case Documentation

Identifier: UC-16
Description: Student accesses physical library materials.
Actor: Student
Precondition: Library membership is valid.
Post condition: Book or material is borrowed.
Basic Course of Action: 1. Search for item. 2. Request borrowing. 3. Return after use.
Alternative Course of Action: Book not available; student queues request.

Table 2.18 Manage Faculty Performance Use Case Documentation

Identifier: UC-17
Description: Instructors administer exams to assess student learning.
Actor: Instructor
Precondition: Course sessions are complete.
Post condition: Exams are conducted.
Basic Course of Action: 1. Prepare exam content. 2. Conduct exam session. 3. Collect answer sheets.
Alternative Course of Action: 1. Exam is postponed.

Table 2.19 Oversee Faculty-Level Academic Affairs Use Case Documentation

Identifier: UC-18
Description: Instructor reviews and grades exam papers.
Actor: Instructor
Precondition: Exam is completed.
Post condition: Grades are recorded.
Basic Course of Action: 1. Review papers. 2. Mark and score based on answer key. 3. Record in system or manually.
Alternative Course of Action: 1. Disputes arise; grades are appealed.

Table 2.20 Enforce University Policies and Quality Standards Use Case Documentation

Identifier: UC-19
Description: Final grades are released to students.
Actor: Instructor/Admin
Precondition: All grading is completed.
Post condition: Students are informed of results.
Basic Course of Action: 1. Compile final grades. 2. Submit to registrar. 3. Publish or distribute to students.
Alternative Course of Action: 1. Errors in grades; corrections made.

Table 2.21 Manage Student Registration Use Case Documentation

Identifier: UC-20
Description: Students request academic transcripts.
Actor: Student
Precondition: Student account is in good standing.
Post condition: Transcript is issued.
Basic Course of Action: 1. Submit transcript request. 2. Registrar prepares transcript. 3. Student receives document.
Alternative Course of Action: 1. Holds on account delay issuance.

Table 2.22 Maintain Academic Records Use Case Documentation

Identifier: UC-21
Description: Student formally withdraws from a registered course.
Actor: Student
Precondition: Course withdrawal period is open.
Post condition: Course is dropped.
Basic Course of Action: 1. Submit withdrawal form. 2. Registrar processes request. 3. Confirmation is sent.
Alternative Course of Action: 1. Deadline passed; withdrawal denied.

Table 2.23 Schedule Classes and Exams Use Case Documentation

Identifier: UC-22
Description: Students update personal contact details.
Actor: Student
Precondition: Student is logged into system or fills paper form.
Post condition: Contact info is updated.
Basic Course of Action: 1. Access/update form. 2. Submit with new data. 3. Admin updates records.
Alternative Course of Action: 1. Admin rejects update due to incorrect data.

Table 2.24 Handle Student Tuition Payments Use Case Documentation

Identifier: UC-23
Description: Admin schedules classrooms for course sessions.
Actor: Administrator
Precondition: Timetable planning is underway.
Post condition: Rooms are booked.
Basic Course of Action: 1. Review course needs. 2. Match with available rooms. 3. Finalize and post schedule.
Alternative Course of Action: 1. Room conflicts; reschedule needed.

Table 2.25 Issue Payment Receipts Use Case Documentation

Identifier: UC-24
Description: Support staff maintain physical infrastructure (classrooms, labs).
Actor: Support Staff
Precondition: Maintenance schedule or report is available.
Post condition: Facilities are functional.
Basic Course of Action: 1. Inspect equipment/facilities. 2. Perform repairs/cleaning. 3. Log completed tasks.
Alternative Course of Action: 1. Delays in maintenance; classes affected.

Table 2.26 Manage Financial Records Use Case Documentation

2.6.4 Essential Flow Diagram

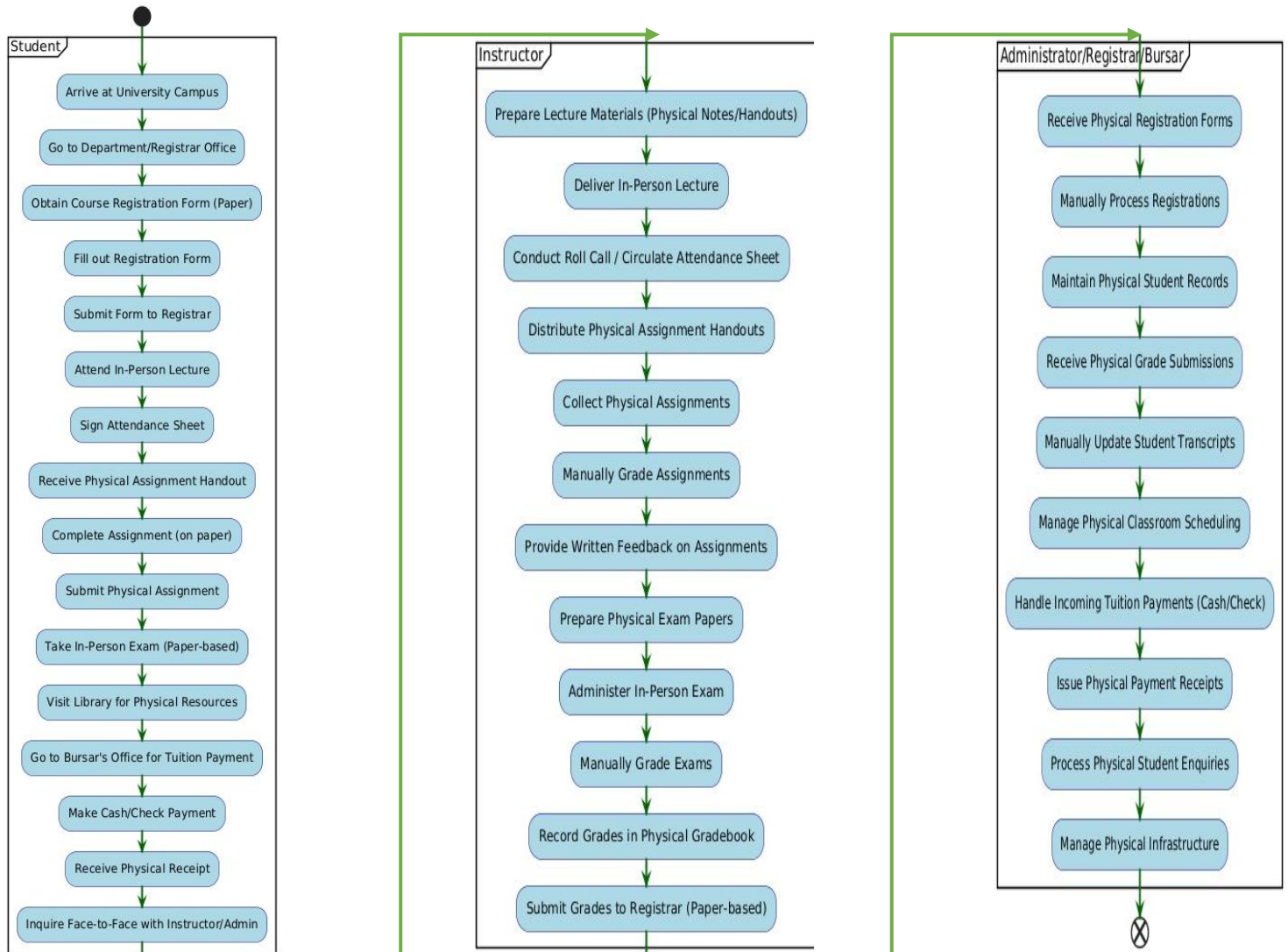


Figure 2.1 Essential Flow Diagram for Existing Diagram

Chapter 3 Proposed system

3.1 Overview of the Proposed New E-learning System

The proposed e-learning system is a comprehensive, standalone web-based platform designed to address the inherent limitations of traditional educational models, as observed in private universities. The primary purpose of this project is to provide a flexible, accessible, and efficient alternative for delivering education, thereby enhancing the overall learning experience for students.

The development of this system is rooted in a careful assessment of the problems prevalent in traditional educational settings. It will offer essential features for online course delivery and management, including secure user accounts for students, instructors, and administrators, allowing for personalized access and management of their respective roles. A robust content management system will enable the uploading, organizing, and delivery of diverse course materials, such as text documents, video lectures, and presentations, ensuring that learning resources are readily available to students anytime, anywhere. The platform will also provide tools for creating and managing quizzes, and assessments, and will feature a system for tracking student progress and performance, generating reports to provide insights into learning outcomes. To foster interaction, the system will incorporate communication features, leveraging Discord for discussion and collaborative activities, facilitating asynchronous communication between students and instructors. Furthermore, a mechanism for handling course enrollment and payment processing will streamline administrative tasks related to student registration and financial transactions. Finally, the system will be built with a responsive design, ensuring optimal accessibility and usability across various devices, including desktops, tablets, and smartphones. By offering these carefully considered features, the new system directly tackles problems such as rigid class schedules, limited resource availability, and the inability to cater to diverse learning paces found in traditional settings. It aims to provide a scalable solution that can serve a large and geographically dispersed student body, promoting a more democratized access to education.

In essence, the proposed e-learning system will be a modern, adaptable, and user-friendly platform that leverages digital technologies to create a more flexible and effective learning environment, supplementing or potentially transforming the educational landscape for students.

3.2. Functional requirement

1. User Authentication and Role Management

- The system allows secure login for three types of users: Students, Instructors, and Administrators.
- Each user type has role-specific access and privileges within the system.

2. Course Management

- Instructors can create, update, and manage courses.
- Students can browse available courses and Enroll as needed.

3. Content Management System (CMS)

- Instructors can upload and organize different types of learning materials (video, and images).
- Students can access course resources anytime.

4. Quiz Management

- Instructors can create and assign quizzes.
- Students can take quizzes within the system.

5. Performance Tracking and Reporting

- The system monitors student progress and records scores.
- Instructors and admins can generate reports on student performance.

6. Communication and Collaboration Tools

- A Discord-integrated communication feature enables chat and discussion.
- Supports asynchronous learning through forums and message boards.

7. Enrollment and Registration Management

- Students can self-Enroll in courses.
- Admins can oversee registration.

8. Payment and Financial Transactions

- Integrated payment gateway allows students to pay for courses.

9. Responsive User Interface

- The system adapts to various devices (desktop, tablet, smartphone).
- Ensures a smooth user experience across screen sizes and platforms.

3.3. Non-functional requirement

➤ Performance
- Our platform delivers a smooth experience under high user loads by simulating traffic, benchmarking key actions such as page loads, and enhancing speed through backend optimization, efficient indexing, and Redis caching.
➤ Security
- User data is protected through comprehensive encryption both at rest and in transit. Strong access controls, including role-based authentication, ensure only authorized users gain access.
➤ Usability
- We prioritize ease of use by conducting real user testing, designing intuitive navigation, and maintaining a consistent design system that facilitates quick learning and seamless interaction.
➤ Scalability
- The system is built on a modular architecture and supports database scalability through sharding and replication, maintaining strong performance as demand increases.

3.4.1.1. Actor Identification for the Proposed System

Based on the functionalities outlined in the "Overview of the Proposed New E-learning System" Canvas, the primary actors who will interact with and utilize the proposed e-learning platform are identified as follows:

No	Actors	Description
1	Student	This actor will be the primary user of the platform, engaging in learning activities. Their responsibilities will include accessing course materials, taking quizzes, tracking their progress, participating in discussions, and managing course enrolment's and payments.
2	Instructor	This actor will be responsible for delivering educational content through the platform. Their responsibilities will include creating and uploading course materials, assigning and grading assessments, monitoring student progress, facilitating discussions, and communicating with students.
3	Administrator	This actor will manage the overall operation and configuration of the e-learning platform. Their responsibilities will include managing user accounts (students and instructors), overseeing course creation and content, generating reports, handling technical support, and potentially managing financial transactions and system settings.

Table 3.1 Identification of Actors

3.4.1.2 Identification of Use Cases

No	Use Case ID	Use Cases
1	UC – 01	User Registration
2	UC – 02	User Login
3	UC – 03	Create Course
4	UC – 04	Enroll in Course
5	UC – 05	Upload Course Materials
6	UC – 06	Access Course Materials
7	UC – 07	Create Quiz
8	UC – 08	Take Quiz
9	UC – 09	Track Student Performance
10	UC – 10	Generate Reports
11	UC – 11	Communication via Discord
12	UC – 12	Manage Enrollment
13	UC – 13	Process Payment
14	UC – 14	View Course Catalog

Table 3.2 Identification of Use Cases

3.4.1.3 System Use Case Diagram

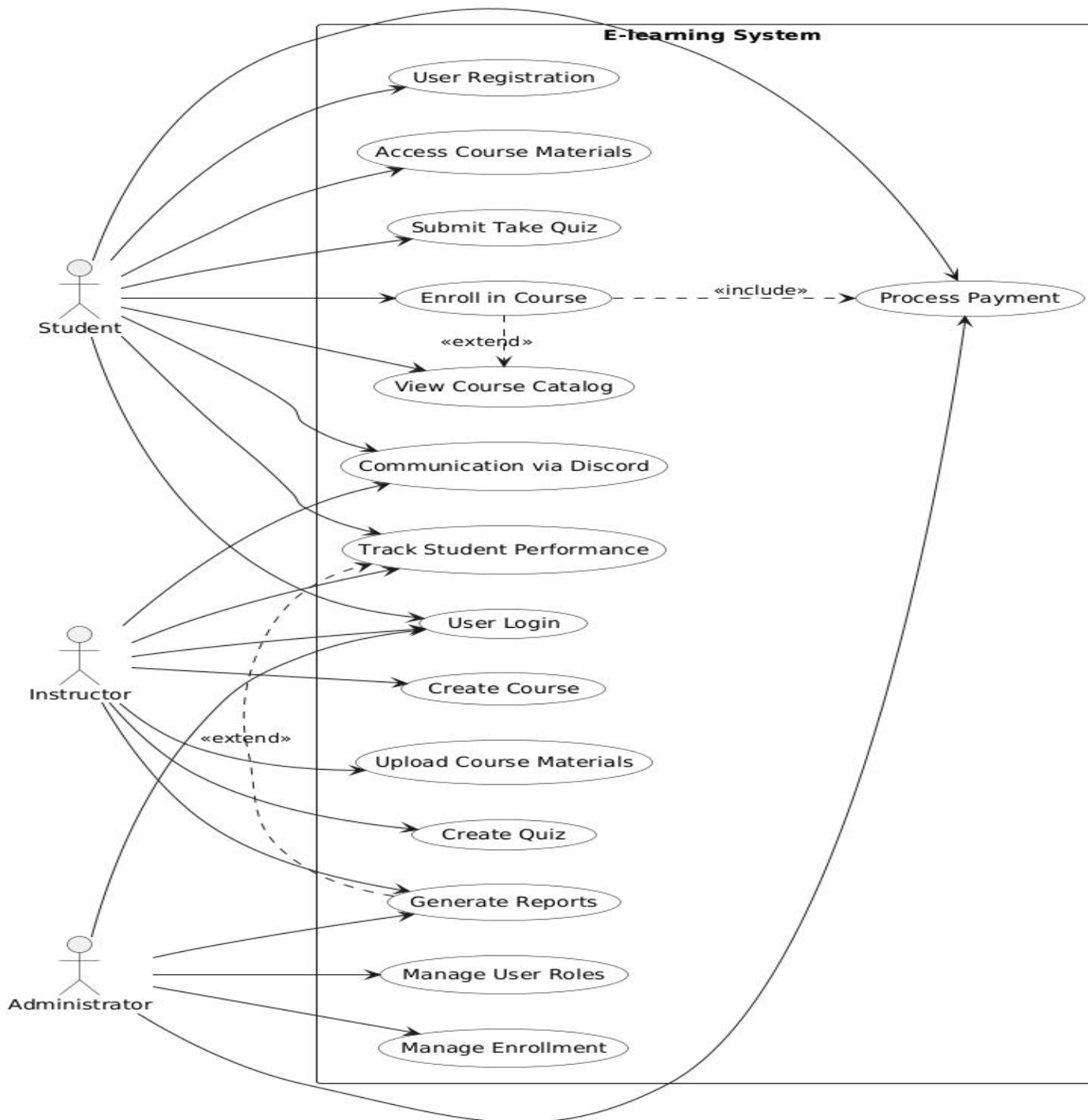


Figure 3.1. Use case diagram

3.4.1.4 System Use Case Documentation

Identifier: UC01
Description: Allows new users to register by providing required personal details.
Actor: Visitor
Precondition: User has not registered.
Post condition: User account is created and stored in the system.
Basic Course of Action: 1. User navigates to the registration page. 2. User fills out the registration form. 3. User submits the form. 4. System validates the input. 5. System stores the data and confirms registration.
Alternative Course of Action: If validation fails, system shows appropriate error message and requests re-entry.

Table 3.3 User Registration

Identifier: UC02
Description: Authenticates users and grants access to the system.
Actor: Registered User
Precondition: User account exists.
Post condition: User is logged into the system.
Basic Course of Action: 1. User accesses the login page. 2. User enters username and password. 3. System validates credentials. 4. User is granted access.
Alternative Course of Action: • If credentials are invalid, the system prompts an error and denies access.

Table 3.4 User Login

Identifier: UC03
Description: Enables instructors to create a new course with relevant details.
Actor: Instructor
Precondition: User is logged in with appropriate privileges.
Post condition: New course is added to the catalog.
Basic Course of Action: 1. Actor accesses the course creation page. 2. Inputs course details. 3. Submits course. 4. Admin validates and stores course.
Alternative Course of Action: Admin notifies user if details are incomplete or invalid.

Table 3.5 Create Course

Identifier: UC04
Description: Enables users to Enroll in available courses.
Actor: Student
Precondition: User is logged in.
Post condition: Enrolment is recorded in the database.
Basic Course of Action: 1. Student views course catalog. 2. Selects a course. 3. Clicks Enroll. 4. System confirms enrollment.
Alternative Course of Action: • If already enrolled, system prevents duplicate entry by Greying or Shadowing the enrol button.

Table 3.6 Enroll in Course

Identifier: UC05
Description: Allows instructors to upload videos and Photo.
Actor: Instructor
Precondition: Instructor will chooses a Category.
Post condition: Materials are available for student access.
Basic Course of Action: 1. Instructor selects course. 2. Uploads material. 3. System stores and links materials to the course.
Alternative Course of Action: • If file type is unsupported, system notifies instructor.

Table 3.7 Upload Course Materials

Identifier: UC06
Description: Enables students to view course materials.
Actor: Student
Precondition: Student is enrolled in the course.
Post condition: Materials are accessed.
Basic Course of Action: 1. Student navigates to enrolled course. 2. Views list of materials.
Alternative Course of Action: • If there is no material the system will show a message to the user to ask for support.

Table 3.8 Access Course Materials

Identifier: UC07
Description: Allows instructors to prepare quizzes for students.
Actor: Instructor
Precondition: Instructor is assigned to a course.
Post condition: Quiz is saved and visible to students.
Basic Course of Action: 1. Instructor accesses Quizz section. 2. Fills out quiz details. 3. Assign to a course. 4. Submits. 5. System saves it.
Alternative Course of Action: • Incomplete details trigger error message.

Table 3.9 Create Quiz

Identifier: UC08
Description: Allows students to submit completed take quizzes.
Actor: Student
Precondition: Quiz is taken
Post condition: Submission is saved in the system.
Basic Course of Action: 1. Student accesses Quizzes. 2. Completes and submits it. 3. System stores submission.
Alternative Course of Action: • If submission is incomplete, system shows warning.

Table 3.10 Submit Take Quiz

Identifier: UC9
Description: Enables instructors to monitor student scores and pprogress.
Actor: Instructor
Precondition: Student has submitted assessments.
Post condition: Performance metrics are visible.
Basic Course of Action: 1. Instructor opens Analytics dashboard. 2. Reviews Course progress.
Alternative Course of Action: • If no new student take a course there will be nothing to monitor.

Table 3.11 Track Student Performance

Identifier: UC10
Description: Allows admin to generate reports on student performance and enrollment report , etc.
Actor: Admin
Precondition: Data is available.
Post condition: Report is generated.
Basic Course of Action: 1. Actor selects report type. 2. System compiles data. 3. Report is generated.
Alternative Course of Action: • If data is insufficient, system informs the actor.

Table 3.12 Generate Reports

Identifier: UC11
Description: Facilitates communication among users through Discord server.
Actor: All Users
Precondition: User is authenticated.
Post condition: User can communicate via Discord.
Basic Course of Action: 1. User clicks communication link. 2. System redirects to Discord server.
Alternative Course of Action: • If Discord integration fails, system shows error.

Table 3.13 Communication via Discord

Identifier: UC12
Description: Enables admin to oversee and manage course Enrollments.
Actor: Admin
Precondition: Courses and users exist.
Post condition: Enrolment statuses are updated.
Basic Course of Action: 1. Admin accesses enrollment section. 2. System updates database.

Alternative Course of Action:

- If there is no enrolment the admin won't take any action.

Table 3.14 Manage Enrollment

Identifier: UC13
Description: Handles online payment for paid courses.
Actor: Student
Precondition: Student selects a course.
Post condition: Payment is processed and access granted.
Basic Course of Action: 1. Student selects course and proceeds to checkout. 2. Enters payment details. 3. System processes payment. 4. Access is granted.
Alternative Course of Action: • Payment failure prompts retry or alternative options.

Table 3.15 Process Payment

Identifier: UC14
Description: Lets users browse available courses.
Actor: Visitor/Student
Precondition: Courses exist in the system.
Post condition: Courses are displayed.
Basic Course of Action: 1. User navigates to catalog page. 2. System displays list of courses. 3. User explores details.
Alternative Course of Action: • If there is no Course the system will show a message to the user to ask for support.

Table 3.16 View Course Catalog

3.4.2. Static Model

3.4.2.1. Class Diagram

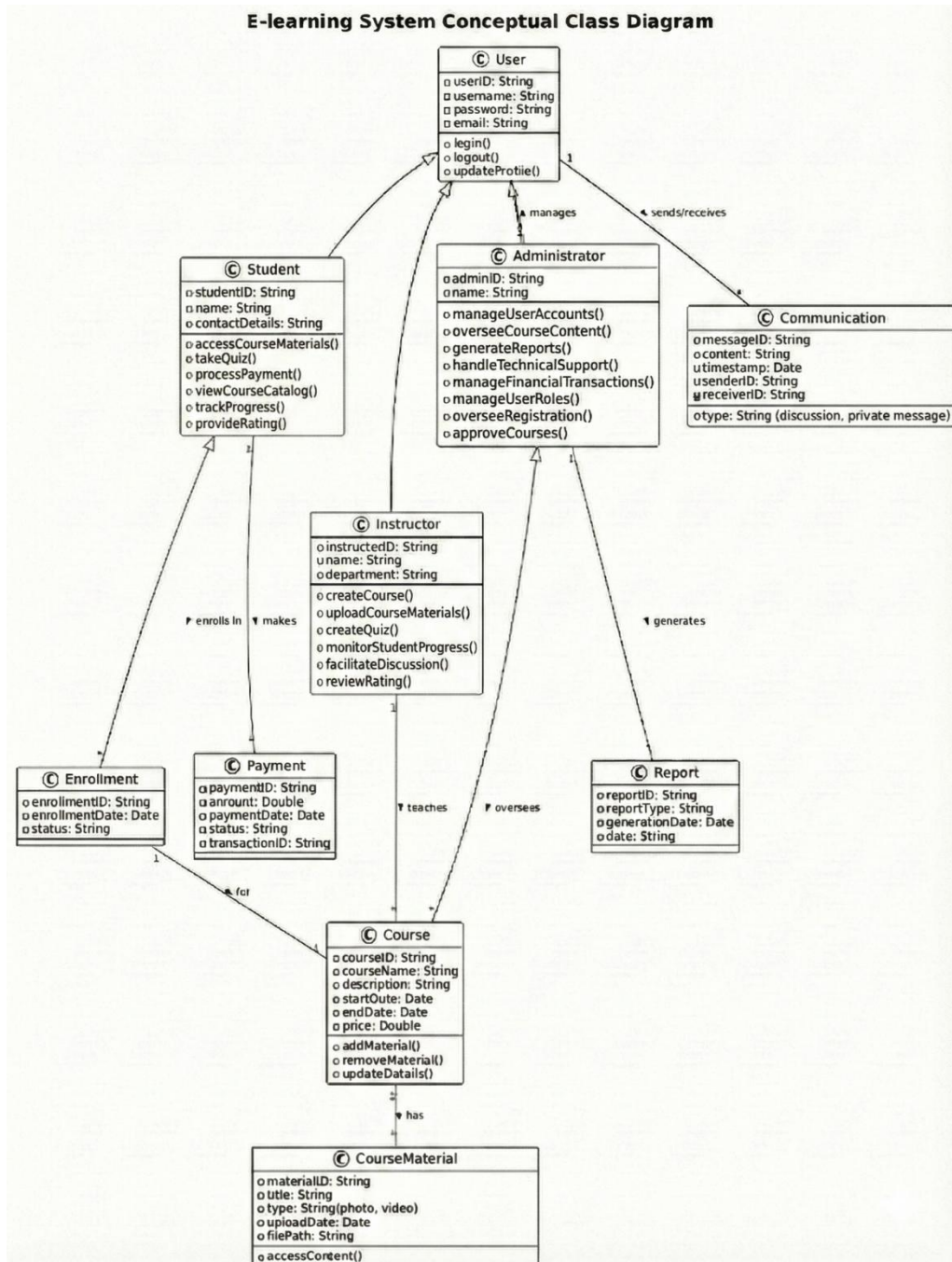


Figure 3.2. Class Diagram

3.4.2.2. Object Diagram

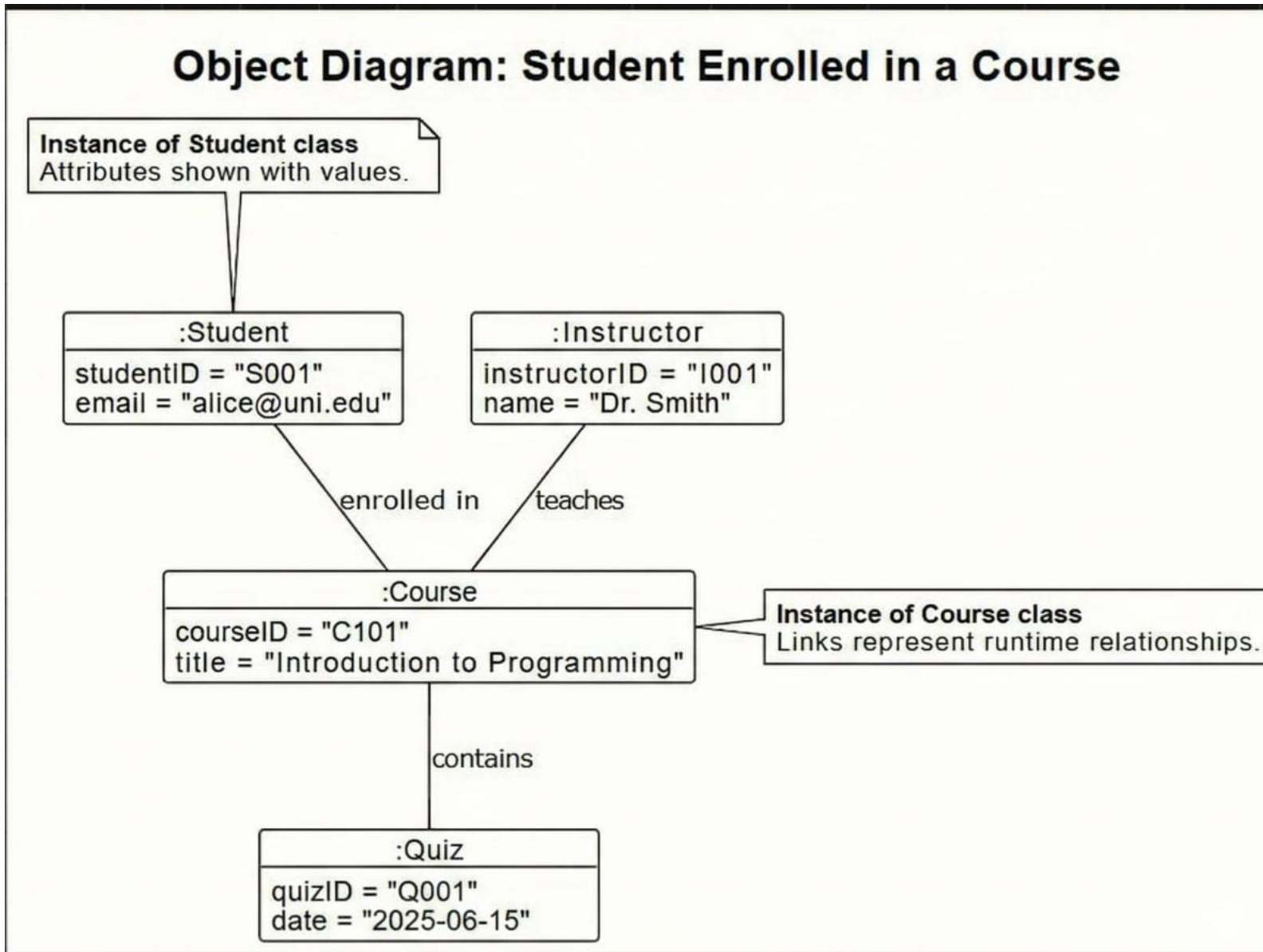


Figure 3.3. Object Diagram Student Enrolled in a Course

3.4.3. Dynamic Model

3.4.3.1. Sequence Diagram

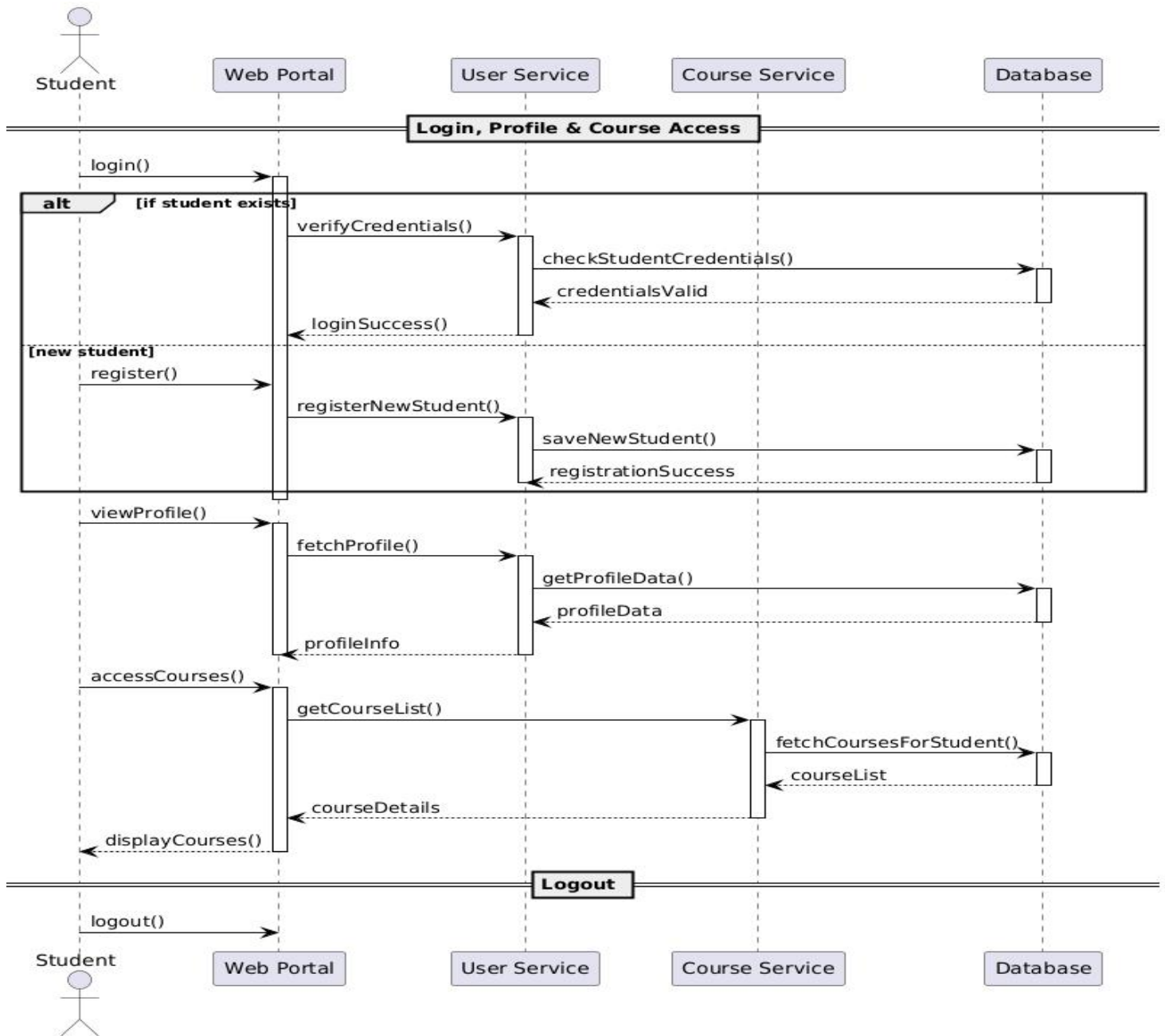


Figure 3.4. Sequence Diagram

3.4.3.2 Activity Diagram

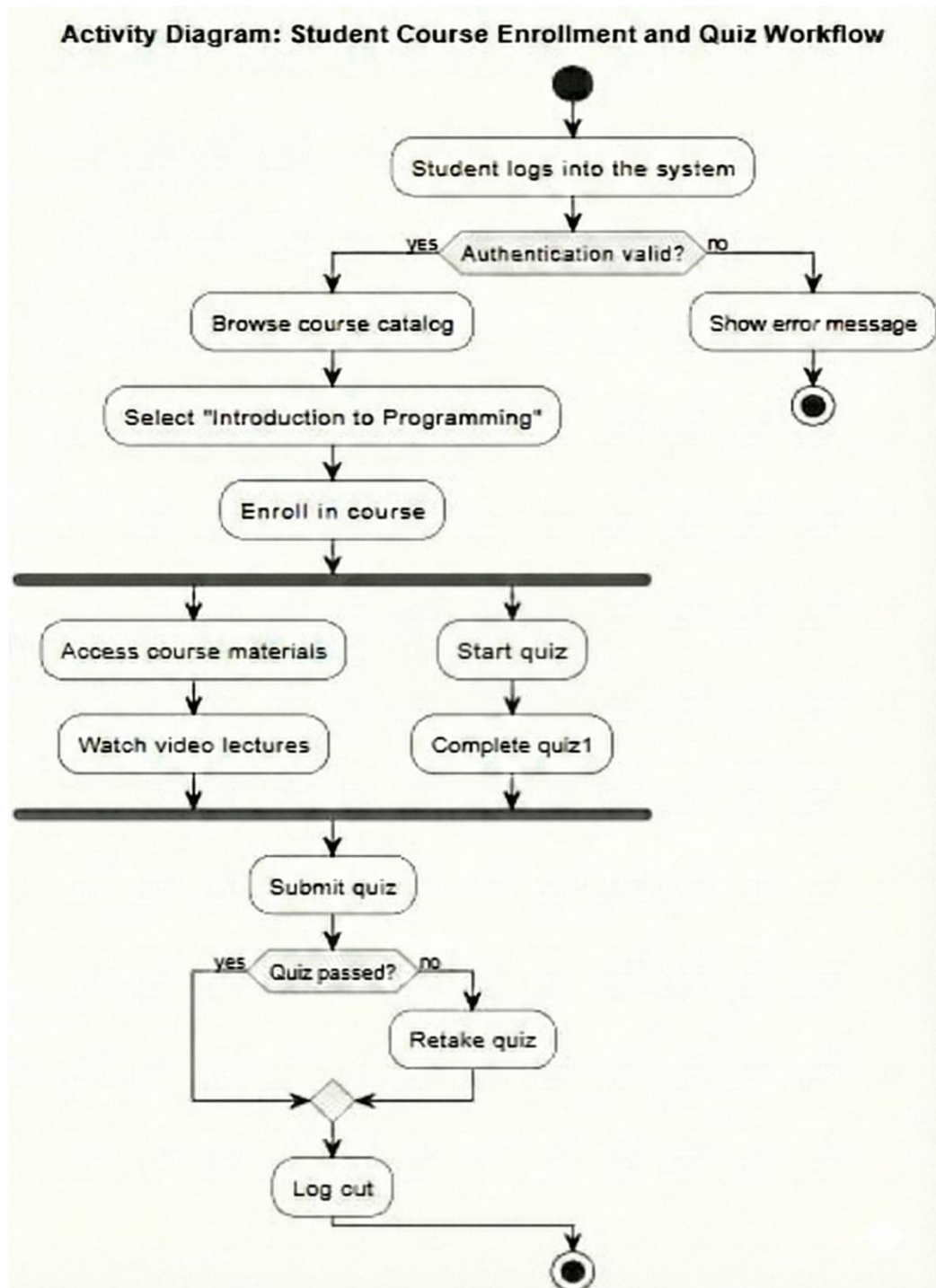


Figure 3.5. Activity Diagram

3.4.3.3. State Diagram

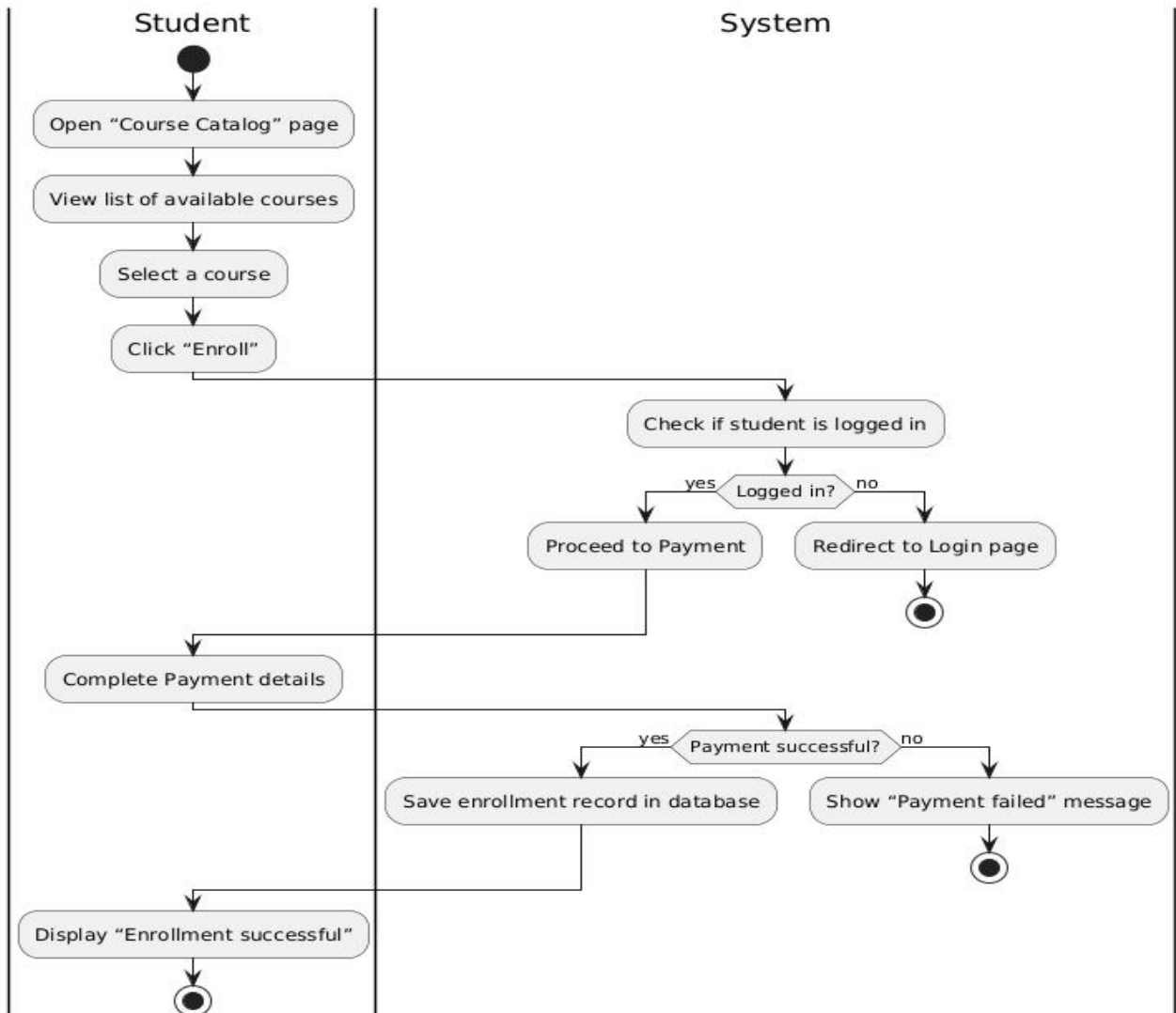


Figure 3.6. State chart Diagram

Chapter 4 System Design

4.1 Overview of System Design

This unit translates your e-learning system's requirements into a technical blueprint using **PostgreSQL** for the database and **React with JavaScript and Tailwind CSS** for the frontend. It defines core design goals—flexibility for diverse course formats, security via role-based access, and scalability to handle student loads—and proposes a three-tier architecture (UI, logic, data) with **React-based interfaces**, **Node.js with Express.js** for backend logic, and **PostgreSQL** for structured data storage using **Prisma ORM** as the database access layer. Subsystems like User Management, Payment Processing (integrated through **Chapa**), and Authentication (using **JWT, bcrypt, and Google OAuth2**) are decomposed, while class diagrams detail methods (e.g., process Payment ()) with visibility modifiers. Collaboration diagrams map object interactions (e.g., Student ↔ Course), and the database schema outlines **PostgreSQL tables** (Students, Courses, Enrollments) with primary/foreign keys. UI wireframes sketch **React-powered screens** (login, catalog), and access control enforces permissions (Student/Instructor/Admin). Pseudocode clarifies algorithms like quiz grading. This plan ensures our **React + Node.js + Express.js + PostgreSQL + Prisma** stack meets all functional needs from Chapter 3.

4.2 Purpose of the System Design

The purpose of the system design is to transform the identified requirements of the proposed e-learning platform into a clear and structured technical framework that guides the actual system development process. This phase acts as a bridge between the analysis (what the system should do) and implementation (how the system will do it).

System design ensures that all components—ranging from user interfaces to backend logic and database structures—are properly planned, coordinated, and aligned with the functional and non-functional requirements defined in Chapter 3. It aims to produce detailed specifications for each system element, reduce ambiguity, and facilitate effective communication among team members during development.

By designing the system architecture, data models, component interactions, and interface layouts, this phase lays the groundwork for building a secure, scalable, and maintainable e-learning platform.

4.3 Design Goal

The system design objective of the e-learning platform is to facilitate a seamless, efficient, and interactive learning experience for both learners and instructors. It is developed to overcome the limitations of traditional learning by offering easy access to educational content, simplified course management, and secure mechanisms for quiz submissions. For instructors, the system provides tools to create and manage courses, monitor student engagement, and offer timely feedback. For administrative personnel, it ensures efficient control over user and course management, access tracking, financial operations, and reporting tools to support institutional decision-making.

To enhance the learning experience, the system adopts a user-centered approach by allowing flexible access to course materials and enabling students to learn at their own pace. It ensures compatibility with various devices and platforms through responsive web design and provides flexible, self-paced learning options. The integration of digital payment gateways further enhances convenience for students enrolling in paid courses. Overall, this design aims to deliver a robust, scalable, secure, and user-friendly solution that accommodates personalized learning experiences while maintaining administrative efficiency and promoting collaborative educational environments.

4.4 System Architecture

For the e-learning system, a multi-tier architecture is adopted, specifically a three-tier architecture. This divides the system into three logical layers: presentation, application (business logic), and data.

Presentation Layer

The presentation layer represents the user interface, built with **React.js and Tailwind CSS**. This layer focuses on user interaction, providing an intuitive interface for students to browse courses and enroll, and for instructors to upload materials. It ensures responsiveness across devices (desktop, tablet, mobile).

Application Layer

The application layer (business logic) handles the core functionalities of the system using Node.js and Express.js. It manages:

- User authentication (login/registration).
- Course enrollment and payment processing via Chapa.
- Quiz grading and progress tracking.
- Communication between the presentation layer and data layer.

Data Layer

- The data layer, powered by PostgreSQL, and managed through prisma ORM, handles storage and retrieval of all system data, including:
- User profiles (students, instructors and admins).
- Course content (videos, images, quizzes).
- Enrollment records and payment transactions.
- It ensures data consistency, security, and efficient querying through indexed tables

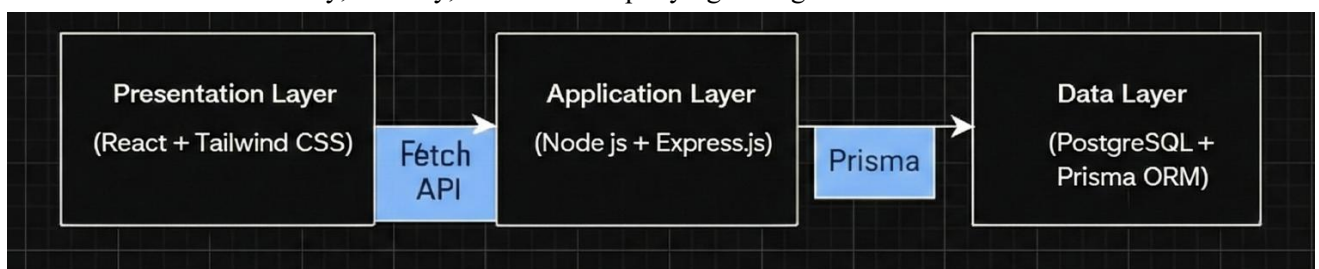


Figure 4.1. Interaction of Layers

4.5 System Process

At the system design level, the e-learning platform operates through a coordinated workflow across its three-tier architecture to deliver seamless educational experiences. Below is the high-level process flow:

User Interaction Initiation

- Students, instructors, or administrators initiate actions (e.g., login, course enrollment, quiz attempts, content upload) through the **presentation layer built with React and Tailwind CSS**.
- Inputs are validated on the client side (e.g., email format, password length) before being sent to the backend through API requests

Business Logic Execution

- The application layer (Node.js + Express.js) receives and processes requests:
- Authenticates users against the PostgreSQL databases.
- Executes role-specific logic (e.g., students enroll, instructors create quizzes).
- Integrates payment gateways for transactions.
- Ensures data consistency (e.g., prevents duplicate enrollments).

Data Management

- The data layer (PostgreSQL + Prisma ORM) performs all CRUD operations:
- Stores/retrieves user profiles, courses, and grades.
- Maintains relational integrity (e.g., foreign keys in enrollments).

Response & Feedback

- The Node.js backend returns results (e.g., login success, quiz score, enrollment status) as **JSON responses**.

The **React frontend** dynamically updates the user interface based on API responses:

- Displaying course access after enrollment.
- Showing quiz results instantly.
- Updating dashboards, progress bars, and notifications.

4.6 System Decomposition

User Authentication and Profile Management Subsystem

- Responsibilities: Manage user registration, login/logout, session handling, and profile updates.
- Dependencies: Needed by all other subsystems for access control.
- Mapping to Hardware: Runs on the server; accessible via frontend on client devices.

- Policy Decisions: Validates credentials, manages session tokens, protects against unauthorized access.

Course Creation and Content Management Subsystem

- Responsibilities: Enables instructors to upload and manage course materials.
- Dependencies: Depends on authentication for role-based access.
- Mapping to Hardware: Server-based logic with frontend upload UI and **PostgreSQL (via Prisma)** for content storage.
- Policy Decisions: Enforces file size/type restrictions, ownership validation.

Discord Communications Subsystem

- Responsibilities: Provides channels (forums), DMs, and announcements for communication.
- Dependencies: Relies on Discord roles for access and the Discord API for all functions.
- Hardware Mapping: Leverages Discord's global servers for handling and client apps for display.
- Policy Decisions: Enforces Discord's message limits and channel access permissions.

Quiz Subsystem

- Responsibilities: Manages quiz creation, delivery, timing, and auto-grading.
- Dependencies: Requires course and user data; uses data from content subsystem.
- Mapping to Hardware: Business logic handled in the **Node.js + Express.js**; results stored in DB.
- Policy Decisions: Enforces quiz lockouts, time limits, and integrity checks.

Progress Tracking and Reporting Subsystem

- Responsibilities: Tracks course completion, grades, quiz scores, and attendance.
- Dependencies: Pulls data from quizzes and enrollments.
- Mapping to Hardware: Server-side processing; reports rendered in UI; data in **PostgreSQL database via Prisma ORM**.
- Policy Decisions: Read-only for students, editable by instructors/admins.

Payment Processing Subsystem

- Responsibilities: Handles course fee transactions.
- Dependencies: Connected to course enrollment logic and third-party payment APIs.
- Mapping to Hardware: Runs on server; payment info stored securely in DB.
- Policy Decisions: Secure transmission, fraud detection, error recover

4.6.1. Subsystem Decomposition

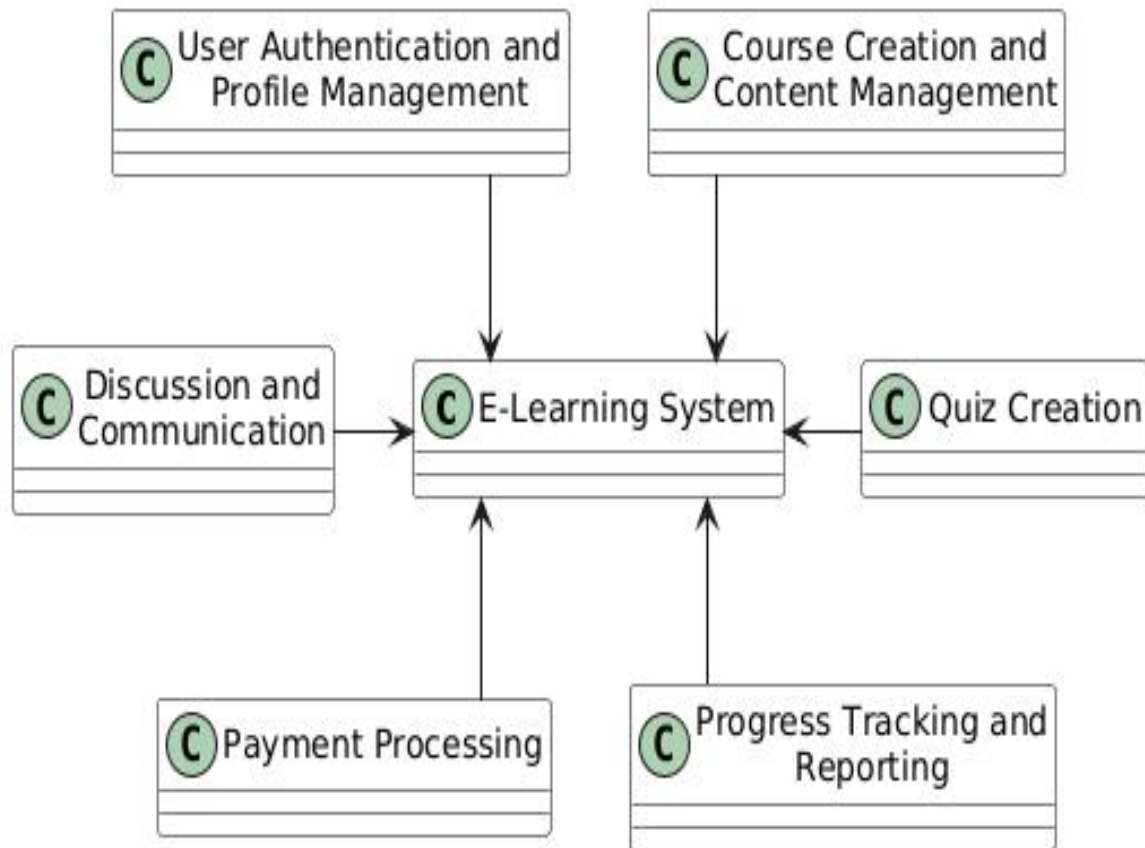


Figure 4.2. Subsystem Decomposition

4.7 Persistent Data Management

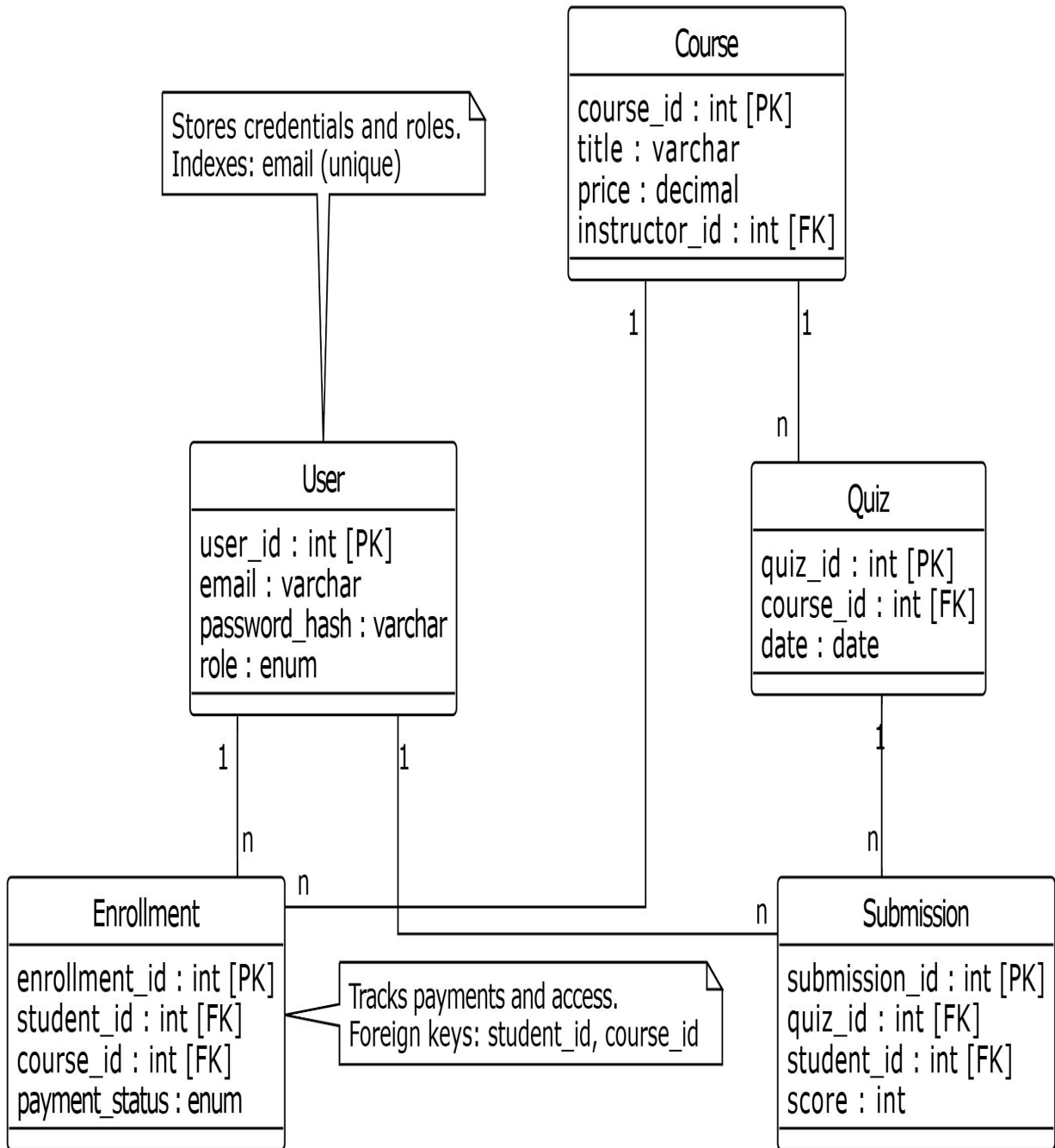


Figure 4.3. Persistent Data Management diagram

4.8 Component Modeling

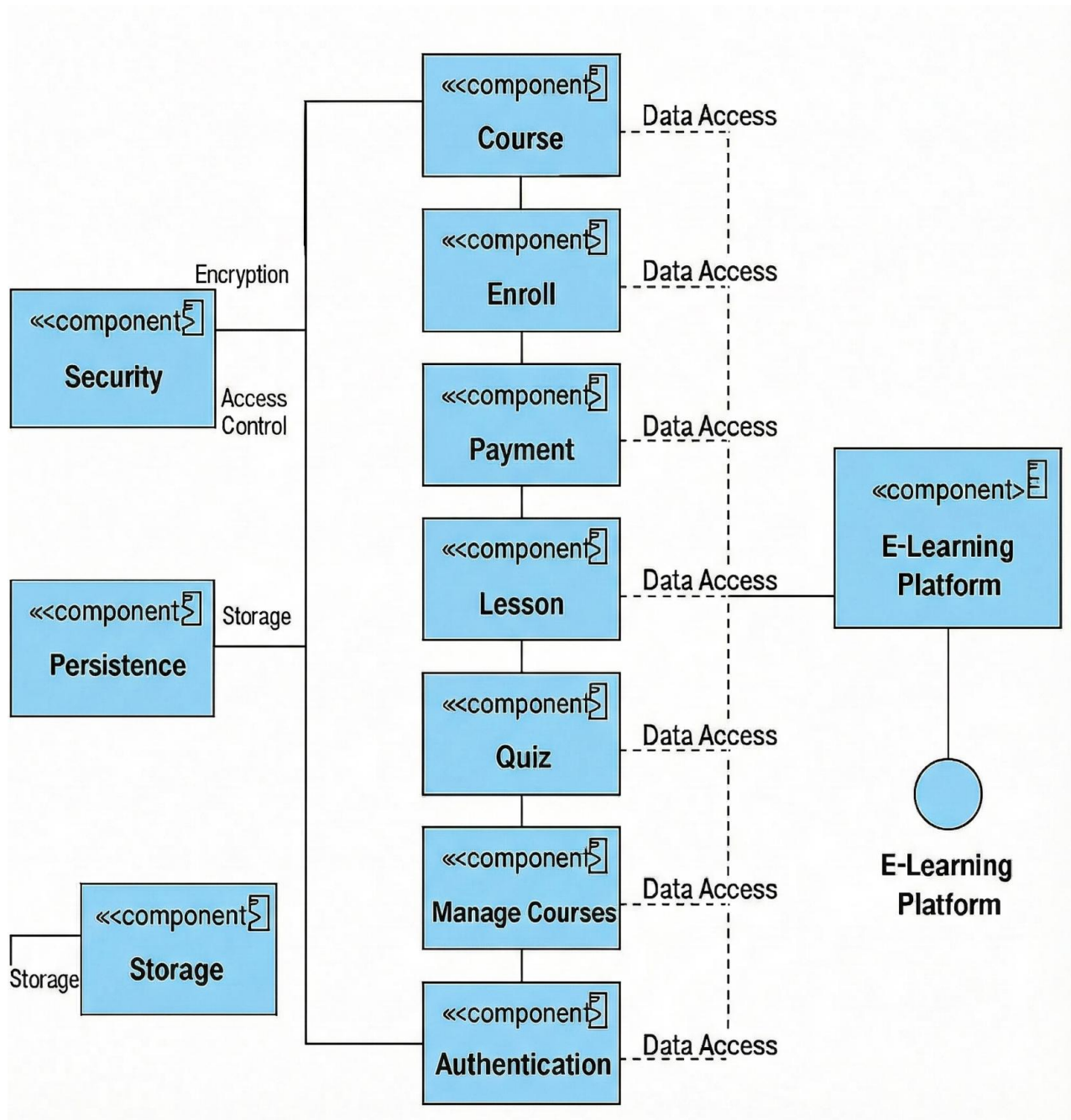


Figure 4.4. Component Modeling

4.9 Deployment Modeling

Deployment modeling allows us to plan and allocate software components to specific hardware resources, such as databases, and client devices. It helps in optimizing resource utilization, ensuring scalability, and improving system performance and reliability.

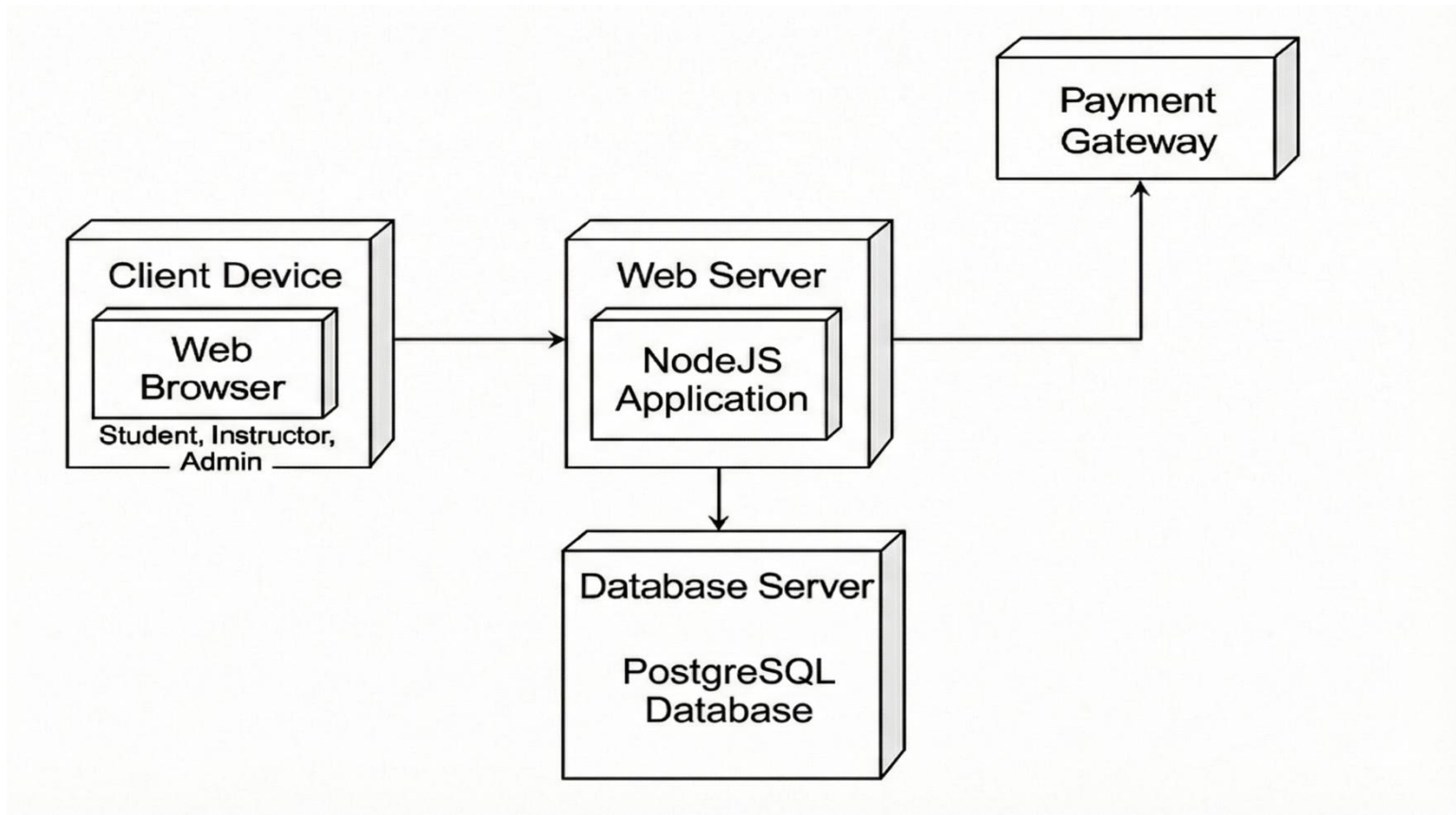


Figure 4.5. Collaboration diagram

4.10. Database design

4.10.1. Database Design Overview

Your e-learning system's database will store:

User data (students, instructors, admins).

Course content (videos, quizzes, images).

Transactions (enrollments, payments).

Quizz records.

Designed for ACID compliance, scalability, and security (e.g., password hashing).

4.10.2. Conceptual Database Design (ER Diagram)

Entities & Relationships:

User → Enrollment ← Course (Many-to-Many). Course → Quiz (One-to-Many).

User ← Quiz (Many-to-Many).

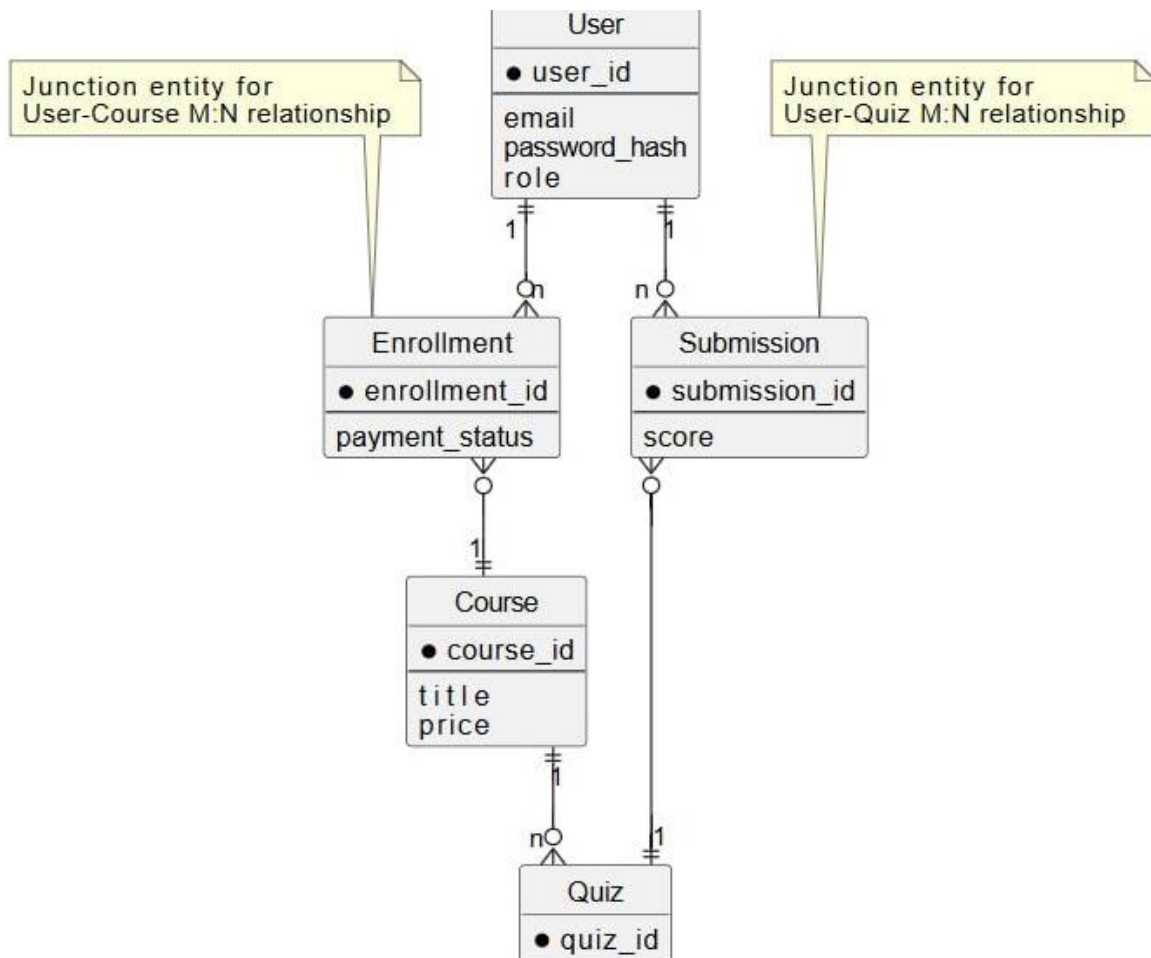


Figure 4.6. Conceptual Database Design

4.10.3. Logical Database Design (Relational Model)

Tables with Foreign Keys:

Users (user_id[PK], email, password_hash, role).

Courses (course_id[PK], title, price, instructor_id[FK]).

Enrollments (enrollment_id[PK], student_id[FK], course_id[FK], payment_status).

Quizzes (quiz_id[PK], course_id[FK], deadline).

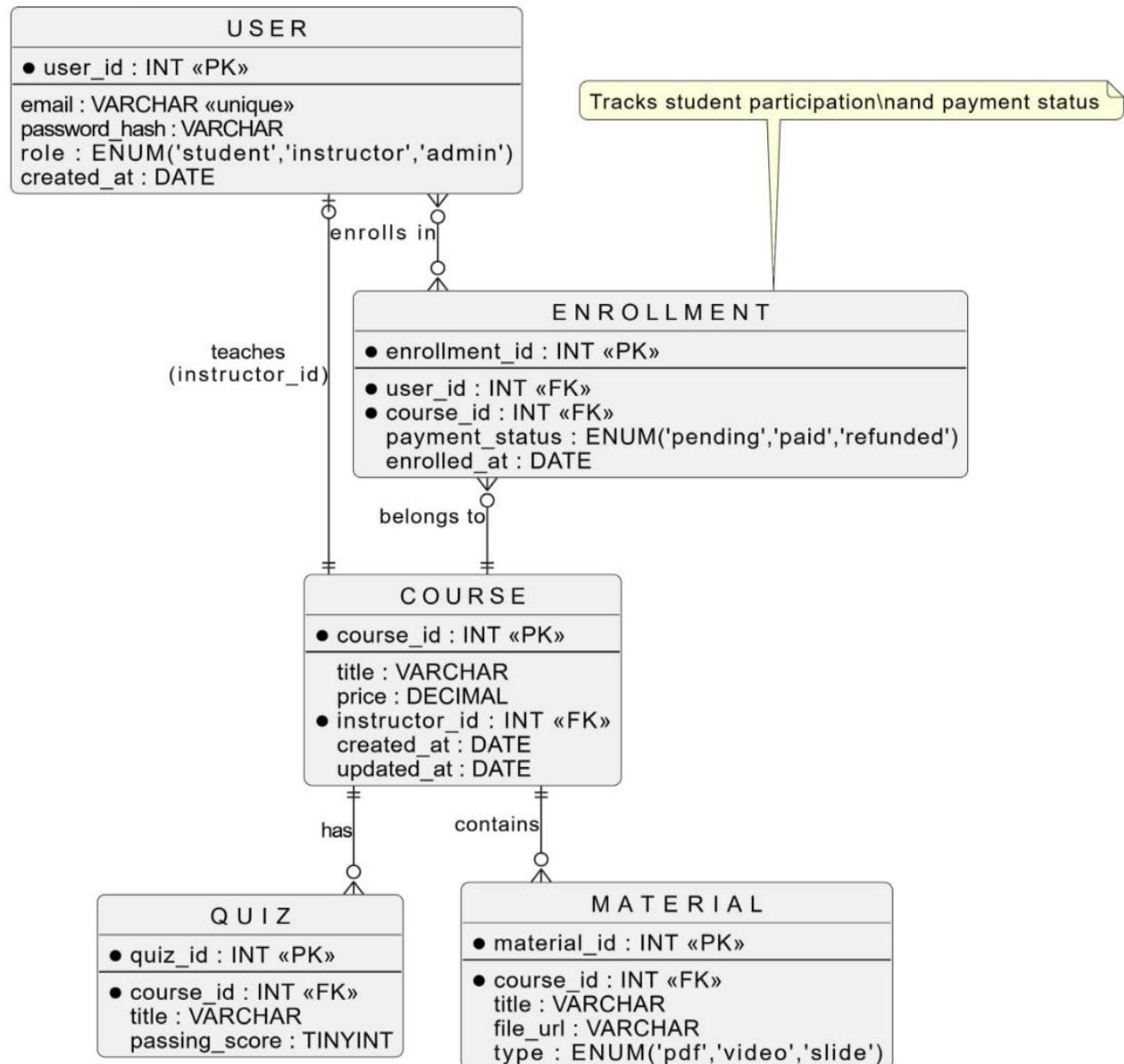


Figure 4.7. Logical Database Design

4.10.4. Physical Database Design

The physical database design defines how data is stored in PostgreSQL, including tables, data types, indexes, and constraints. Below is a concise tabular representation of the PostgreSQL realization.

Enrollments Table

Table	column	Data Type	Constraints	Index	Description
Enrollment	Id	SERIAL	PRIMARY KEY	PRIMARY	Unique enrollment ID
	Student_id	INT	NOT NULL, FOREIGN KEY → Users(id)	idx_student	Student enrolled in the course
	course_id	INT	NOT NULL, FOREIGN KEY → Courses(id)	idx_course	Course being enrolled in
	status	VARCHAR(50)	DEFAULT 'pending'	-	Enrollment/payment status
	payment_ref	VARCHAR(50)	NULLABLE	-	Payment reference code
	created_at	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP	-	Enrollment creation timestamp

Table 4.1 Physical Database Design

Key Physical Design Choices

Data Types:

INT for IDs, VARCHAR(255) for text, DECIMAL(10,2) for currency.

TIMESTAMP for auto-tracked timestamps.

Indexes:

Primary: All _id columns.

Secondary: Foreign keys (user_id, course_id) and searchable fields (email, title).

Security:

password_hash uses BCrypt for encryption.

role restricts access via application logic.

Example data representation

Sample Data Representation

users			
user_id	email	role	created_at
1	student@edu.com	student	2025-06-01 10:00:00
2	prof@uni.com	instructor	2025-06-01 10:10:00

courses			
course_id	title	price	instructor_id
101	Intro to Python	50.00	2
102	Data Structures	0.00	2

enrollments			
enrollment_id	user_id	course_id	payment status
1001	1	101	completed
1002	1	102	pending

quizzes		
quiz_id	course_id	deadline
501	101	2025-06-10 23:59:00
502	102	2025-06-15 23:59:00

Figure 4.8. Data Representation Design

4.11 Access Control for an E-Learning Platform

In designing the access control mechanism for the E-Learning Platform, it is essential to ensure that only authorized users can perform specific actions relevant to their roles within the learning environment.

Here's a short description of the access control mechanism with examples:

Role-Based Access Control (RBAC):

Description: The system categorizes users into distinct roles, and each role is assigned a specific set of permissions that dictates what actions they can perform and what resources they can access.

Examples:

Student Role: Can view enrolled courses, access course materials and take quizzes

Instructor Role: Can create and manage their courses, upload course materials, design quizzes, and view student progress within their courses. Cannot manage other instructors' courses or access system-wide settings.

Administrator Role: Has full system-wide access, including managing all courses, users (students and instructors), and viewing comprehensive analytics.

Guest Role (Unauthenticated User): Can browse public course catalogs, view introductory course details, and register for an account. Cannot access specific course content or submit anything.

Authentication and Authorization:

Description: Users are required to verify their identity (authenticate) before gaining access to the system. Once authenticated, authorization is then granted or denied based on their assigned role and the specific permissions associated with that role.

Examples:

Users must successfully log in with a valid username (e.g., email) and password via the Authentication Component.

After logging in, a customer cannot access the administrator dashboard due to role-based restrictions.

An instructor can access the "Course Creation" feature only for courses they are assigned to teach.

These access control mechanisms collectively contribute to a secure and controlled learning environment within the E-Learning Platform, ensuring that user actions align with their assigned roles and that sensitive data (like personal information, or course content) is safeguarded against unauthorized access.

4.12. User interface design

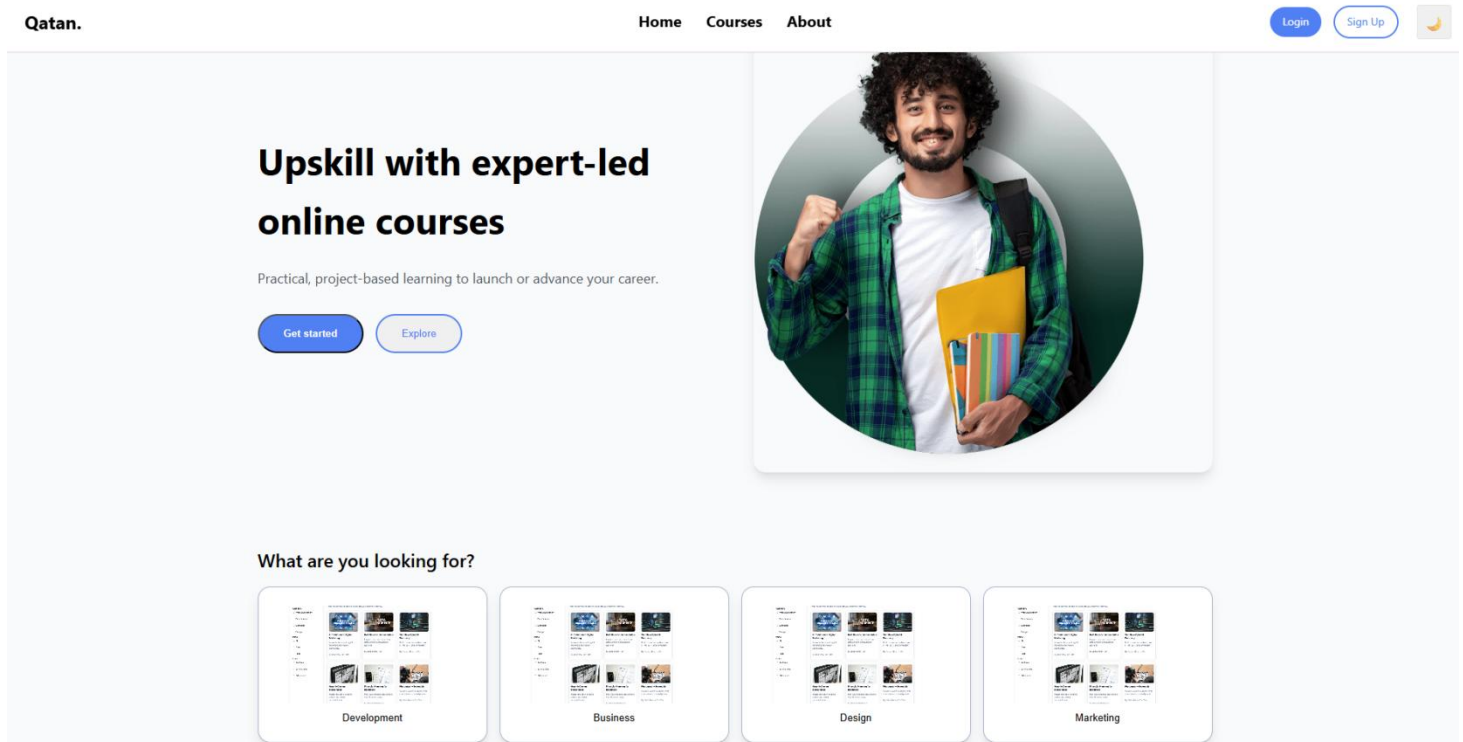


Figure 4.9 Homepage

Filters

Category

[All](#)[Design](#)[Development](#)[Marketing](#)[Business](#)

Price

[All](#)[Free](#)[Paid](#)

Courses

**Business Communication & Leadership Skills**

Eden

Rating:

\$123

Business

**Entrepreneurship & Small Business Management**

Eden

Rating: 4.0 ★ (1)

Free

Business

**Social Media Marketing for Business Growth**

Nahom

Rating: 4.5 ★ (2)

Free

Marketing

**Web Development**

Eden

Rating:

Free

Development

**Entrepreneurship & Small Business Management**

Nahom

Rating:

**Python**

Nahom

Rating: 3.5 ★ (2)

Free

Development

Figure 4.10 Course Page

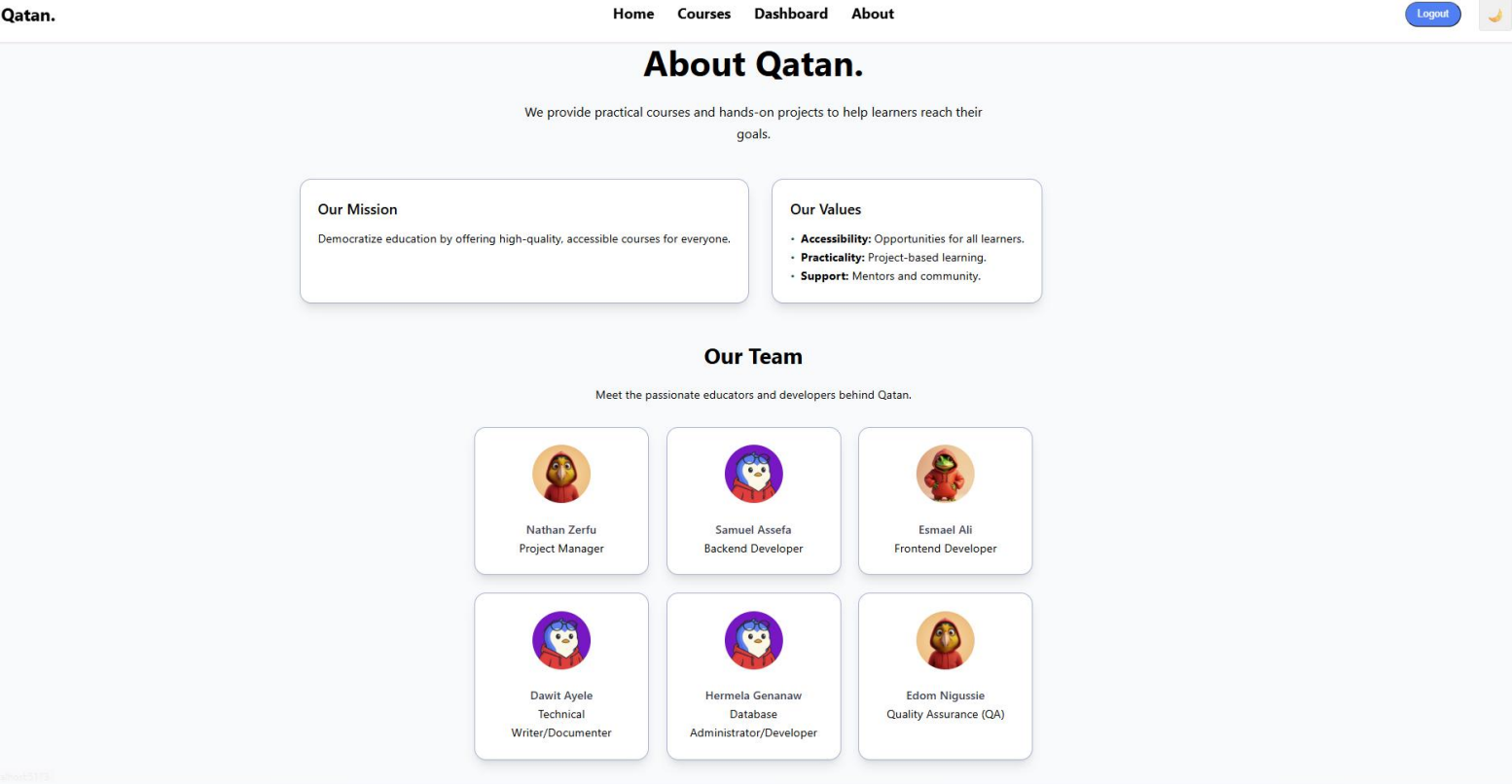


Figure 4.11 about Us Page

CHAPTER FIVE: IMPLEMENTATION AND TESTING

5.1 Introduction

This chapter describes how the Qatan E-Learning System was implemented using modern web technologies and how it was tested to ensure quality and reliability. After completing the design phase, the system was developed using a full-stack JavaScript architecture consisting of React for the front-end, Node.js with Express for the backend, and PostgreSQL as the database, managed through Prisma ORM.

The implementation process translated design specifications into functional code, following clean coding practices and modular structures to ensure readability, maintainability, and scalability.

The chapter also details the testing approaches used to verify correctness, identify errors, check performance, and validate that the system meets the functional requirements defined earlier.

5.2 Coding

The coding stage focused on transforming the full system design into functional code. To build a system that is easy to understand, maintain, and improve in the future, several coding principles were followed.

5.2.1 Coding Style

To ensure high-quality code and maintainability, the following coding standards were applied:

Frontend (React + JavaScript + Tailwind CSS):

- Functional components with React Hooks
- Consistent **camel Case** naming convention
- Reusable components (buttons, cards, forms)
- Clean file structure: /components, /pages, /services
- Tailwind utility classes for styling
- Comments for complex logic and API calls
- Avoiding duplicated logic via custom hooks and helpers

Backend (Node.js + Express):

- Use of **ES6+** syntax (let/const, arrow functions, async/await)
- Module-based routing: /auth, /courses, /quiz, /enrollment
- Middleware for authentication and error handling
- Consistent HTTP response formatting (JSON API standards)
- Input validation using middleware
- Clear separation of controllers, services, and routes

Database (PostgreSQL + Prisma ORM):

- Clean schema definitions in schema.prisma
- Meaningful field names and enums
- Referential integrity using foreign keys
- Migration-based development using prisma migrate
- Using Prisma Client for safe database queries

These practices improved readability, made debugging easier, and prepared the system for future expansions.

5.2.2 Programming Language and Technology Selection

The system was implemented using the following technologies chosen for their reliability, scalability, and suitability for modern web applications:

1. Node.js + Express (Backend)

- Provides fast, scalable, non-blocking API performance
- Supports building RESTful APIs easily
- Large ecosystem of packages
- Ideal for handling asynchronous operations such as authentication, content loading, and quiz processing

2. React (Frontend)

- Component-based structure allows reusability and modularity
- Efficient UI rendering using the virtual DOM
- Works seamlessly with Tailwind CSS for responsive design

3. Tailwind CSS

- Utility-first CSS framework
- Enables rapid UI development
- Ensures consistent styling

4. PostgreSQL (Database)

- Reliable relational database
- Ensures data integrity through relations
- Suitable for storing structured data such as users, courses, quizzes, and results

5. Prisma ORM

- Simplifies database operations
- Auto-generates type-safe queries
- Provides migrations and schema management
- Improves security by preventing raw SQL vulnerabilities

6. Axios + Fetch API

- Used for connecting frontend to backend through API requests
- Supports asynchronous data flows

These technologies together allowed us to build a modern, responsive, scalable e-learning platform.

5.3 Software Testing and Procedures

Software testing was performed to ensure the system works correctly, meets user expectations, and handles real-world scenarios without failure. Testing focused on:

- Identifying **errors**
- Checking **requirement conformance**
- Evaluating **performance**
- Ensuring **overall quality**

Testing verified both **validation** ("Are we building the right system?") and **verification** ("Are we building the system right?").

5.3.1 Software Testing

During testing, the following key components were evaluated:

- User authentication (login, registration, session handling)
- Course creation and management
- Enrollment and access control
- Material upload and retrieval
- Quiz creation, submission, and scoring
- Dashboard and progress tracking
- Database consistency using Prisma queries
- API route correctness and error handling

The testing confirmed that the system meets the expected behavior for students, instructors, and administrators.

5.3.2 Testing Procedures

Three major levels of testing were conducted:

1. Unit Testing

Individual components and API endpoints were tested separately to ensure they operate correctly on their own.

Examples:

- Testing login API with valid/invalid credentials
- Verifying course creation API using Postman
- Checking quiz creation and JSON responses
- React component rendering tests
- Prisma queries returning correct database results

Unit testing helped detect issues early in development.

2. Integration Testing

Modules were tested together to ensure smooth data flow and interaction.

Examples:

- Frontend calling backend APIs
- Enrollment → database update → dashboard update
- Instructor creates quiz → student accesses quiz → backend saves submission
- Material uploaded → saved in DB → visible in student UI

Integration testing ensured all parts of the system communicate correctly.

3. System Testing

The entire application was tested end-to-end, simulating real user behavior.

This testing verified:

- Full user journeys (login → enroll → access content → take quiz)
- Page navigation and responsiveness
- Access control for different roles
- Database accuracy (enrollments, quiz results, progress)
- Fast loading on both desktop and mobile
- Frontend and backend error messages

System testing validated that the platform is ready for actual usage.

CHAPTER SIX: SUMMARY, CONCLUSION AND RECOMMENDATION

6.1 Summary and Conclusion

This project involved the design and development of the Qatan E-Learning System, a web-based platform aimed at improving teaching and learning processes. The system addresses challenges found in traditional and semi-digital learning environments, such as scattered learning materials, limited resource access, inefficient course management, and the difficulty of tracking student progress. Through a systematic approach, the project analyzed user requirements, designed a three-tier system architecture, and implemented features including course management, quizzes, and user progress tracking.

Using modern technologies—React for the front-end, Node.js with Express for the backend, and PostgreSQL with Prisma ORM for database management—the system was built, tested, and verified for functionality through unit, integration, and system testing. In conclusion, the Qatan E-Learning System successfully provides a structured, interactive, and accessible learning environment that enhances communication between students and instructors, streamlines course delivery, and demonstrates how digital solutions can improve educational quality and accessibility.

6.2 Recommendation

Based on the development and evaluation of the system, the following recommendations are suggested to enhance the platform in future versions:

- **Add advanced analytics** for instructors, such as student performance dashboards and course engagement tracking.
- **Integrate video streaming or live class features** to support synchronous learning in addition to the current asynchronous model.
- **Implement mobile applications** (Android/iOS) to make the system more accessible for students with smartphones.
- **Enhance security** by adding two-factor authentication, stronger password rules, and improved admin-level controls.
- **Include multilingual support** to help users who prefer learning in different local languages.
- **Expand the admin panel** to allow more flexible management of courses, content, and system configurations.

Implementing these improvements will make the Qatan E-Learning System more powerful, scalable, and user-centered.

6.3 Reference

- **React Documentation. (n.d.).** Reference for developing dynamic and interactive user interfaces in React. Retrieved from <https://react.dev/>
- **Node.js Documentation. (n.d.).** Comprehensive guide for building scalable server-side applications using JavaScript. Retrieved from <https://nodejs.org/docs/latest/api/>
- **Express.js Documentation. (n.d.).** Resource for creating robust backend APIs and server-side applications. Retrieved from <https://expressjs.com/>
- **Tailwind CSS Documentation. (n.d.).** Guide for building responsive and modern user interfaces using utility-first CSS. Retrieved from <https://tailwindcss.com/docs>
- **Prisma ORM Documentation. (n.d.).** Reference for modern database access, schema modeling, and migrations. Retrieved from <https://www.prisma.io/docs>
- **PostgreSQL Documentation. (n.d.).** Guide to relational database management and SQL operations. Retrieved from <https://www.postgresql.org/docs/>
- **MDN Web Docs. (n.d.).** HTML reference for structuring web pages. Retrieved from <https://developer.mozilla.org/en-US/docs/Web/HTML>
- **MDN Web Docs. (n.d.).** CSS reference for styling and layout design. Retrieved from <https://developer.mozilla.org/en-US/docs/Web/CSS>
- **MDN Web Docs. (n.d.).** JavaScript documentation for frontend and backend scripting. Retrieved from <https://developer.mozilla.org/en-US/docs/Web/JavaScript>
- **Axios Documentation. (n.d.).** Guide for handling HTTP requests in frontend and backend applications. Retrieved from <https://axios-http.com/docs/intro>

Appendixes

Sample Code Snippets

1. Database Schema for Users Enrollment

```
model Enrollment {  
  id      Int    @id @default(autoincrement())  
  studentId Int  
  courseId Int  
  status   String @default("pending")  
  paymentRef String?  
  createdAt DateTime @default(now())  
  course   Course  @relation(fields: [courseId], references: [id])  
  student  User    @relation(fields: [studentId], references: [id])  
}
```

2. contact

```
/* src/routes/contact.js */  
  
import express from "express";  
  
import {  
  sendMessage,  
  getGmailAuthUrl,  
  gmailOAuthCallback,  
} from "../controllers/contactController.js";  
  
const router = express.Router();  
  
// POST /api/contact/send  
router.post("/send", sendMessage);  
  
// Gmail OAuth endpoints  
  
// GET /api/contact/gmail/auth-url -> returns { authUrl }  
router.get("/gmail/auth-url", getGmailAuthUrl);  
  
// GET /api/contact/gmail/callback -> Google redirects here after consent  
router.get("/gmail/callback", gmailOAuthCallback);  
  
export default router;
```